



Programa Profesional en Inteligencia Artificial y Data
Science

Módulo 2. Lenguajes y desarrollo de soluciones IA

Índice

Tema 1. Programas informáticos y lenguajes de programación

1.1. Introducción y objetivos

1.2. Algoritmos

Notación para cada posible acción

1.3. Pseudocódigo

1.4. Programas informáticos

1.5. Lenguajes de programación

Tema 2. Características de los lenguajes de programación para IA

2.1. Introducción y objetivos

2.2. Evolución de los lenguajes de programación

2.3. Sintaxis y estructura de los lenguajes de programación

2.4. Paradigmas de programación

2.5. Características de los lenguajes de programación para IA

Tema 3. Principales lenguajes de programación para IA

3.1. Introducción y objetivos

3.2. Lenguajes de programación para IA

3.3. Python

3.4. R

3.5. Java

3.6. Lisp

3.7. JavaScript

3.8. Prolog

3.9. Otros lenguajes y tecnologías

Tema 4. Lenguajes de marcado. Información de sus etiquetas

4.1. Introducción y objetivos

4.2. Los lenguajes de marcado

4.3. Tipos de lenguajes de marcado

4.4. HTML

4.5. XML

4.6. XHTML

4.7. LaTeX

4.8. Markdown

4.9. Otras tecnologías

Tema 1. Programas informáticos y lenguajes de programación

1.1. Introducción y objetivos

Para estudiar este tema lee las Ideas Clave que se presentan a continuación y revisa los contenidos adicionales para mejorar la comprensión y aumentar los conocimientos sobre las materias tratadas.

Este tema supone la primera introducción a las bases de la programación, los algoritmos, los programas y los lenguajes de programación. Objetivos más concretos son:

- ▶ Conocer qué son los algoritmos y el papel que juegan en la construcción de aplicaciones software.
- ▶ Tener un acercamiento a ejemplos reales de pseudocódigo.
- ▶ Conocer cómo se representan visualmente los algoritmos por medio de diagramas.
- ▶ Aplicar las bases del pensamiento computacional.
- ▶ Categorizar los distintos tipos de programas software.
- ▶ Conocer algunos de los términos principales en el ámbito.

Tema 1. Programas informáticos y lenguajes de programación

1.2. Algoritmos

La palabra algoritmo, uno de los términos tecnológicos de moda, frecuentemente, es usado con cierto desconocimiento cuando hablamos de la **inteligencia artificial** o de conceptos cercanos.

Un **algoritmo** es, sencillamente, una secuencia ordenada y finita de operaciones sencillas a través de la cual podemos determinar la solución a un problema.

Los algoritmos nos permiten ejecutar una **acción** o **resolver un problema** mediante una serie de instrucciones definidas, ordenadas y finitas.

Basándonos en esta definición, tanto una receta de cocina como el cálculo de una raíz cuadrada de un número o un protocolo sanitario concreto son algoritmos.

Un algoritmo es una secuencia de instrucciones finitas que llevan a cabo un conjunto de procesos de forma lógica, sistemática y ordenada para dar respuesta a un determinado problema o propósito, con lo que un algoritmo resuelve un problema concreto a través de una serie finita de una o más instrucciones definidas, concisas y ordenadas.

Características de los algoritmos

Las **características** básicas de un algoritmo son:

- ▶ **Serie finita:** tiene inicio y fin, todo algoritmo comienza en un estado inicial con una serie de datos específicos, y culmina con una solución o salida.
- ▶ **Secuencialidad:** un algoritmo está compuesto por una serie de pasos ordenados.
- ▶ **Sin ambigüedad:** las secuencias son concretas, cada paso es claro y no deja lugar a ambigüedad.

Tema 1. Programas informáticos y lenguajes de programación

- ▶ **Validez.** Un algoritmo es válido si carece de errores. Un algoritmo puede resolver el problema para el que se planteó y sin embargo no ser válido debido a que posee errores.
- ▶ **Eficiencia.** Un algoritmo es eficiente si obtiene la solución al problema en poco tiempo. No lo es si es lento en obtener el resultado.
- ▶ **Optimización.** Un algoritmo es óptimo si es el más eficiente posible y no contiene errores. La búsqueda de este algoritmo es el objetivo prioritario del programador. No siempre podemos garantizar que el algoritmo hallado es el óptimo, a veces sí.
- ▶ Ser **independiente** de la computadora. Los algoritmos deben escribirse para poder ser utilizados por cualquier máquina.
- ▶ Ser **preciso**. Los resultados de los cálculos deben de ser exactos, no solo aproximarse a la solución.
- ▶ Debe poder **repetirse**. Los algoritmos deben permitir su ejecución tantas veces como sea necesario, además de resolver el problema cambiando los datos de entrada.

Condiciones que deben cumplir los algoritmos:

- ▶ **Finitud** del número de acciones elementales.
- ▶ **Definibilidad.** Cada paso debe efectuarse de forma precisa. Precisión en el lenguaje: pseudocódigo.
- ▶ **Entrada.** Deben estar especificados los datos a los que se puede aplicar.
- ▶ **Salida.** Debe especificarse el conjunto de todas las salidas que pueden producirse.
- ▶ **Efectividad.** Todos los procesos deben ejecutarse en un tiempo finito.

Tema 1. Programas informáticos y lenguajes de programación

Algoritmia

La **algoritmia** es el tratamiento sistemático de técnicas para el diseño y análisis de algoritmos eficientes, que faciliten el desarrollo de programas de ordenador.

Comprende:

- ▶ Estudio conceptual
- ▶ Criterios de evaluación

Estudio conceptual apoyándose en:

- ▶ La modelización de los datos (árboles, grafos, ...)
- ▶ Los modelos de máquinas (Turing, Von Neumann).
- ▶ Los métodos (divide y vencerás, recursión, ...)
- ▶ Estudio algoritmos clásicos en modelos discretos (palabras, grafos, ...)

Conocimiento **criterios de evaluación**:

- ▶ Las prestaciones de los algoritmos (complejidad).
- ▶ Su corrección (verificación).
- ▶ Su expresión en un lenguaje adecuado (Java, C, Python...)

Visualización de algoritmos

Los **diagramas de flujo** son representaciones gráficas que muestran los pasos de un proceso. Permiten representar procesos computacionales con el fin de comunicar y debatir sobre los mismos en las etapas previas al desarrollo de un código.

Tema 1. Programas informáticos y lenguajes de programación

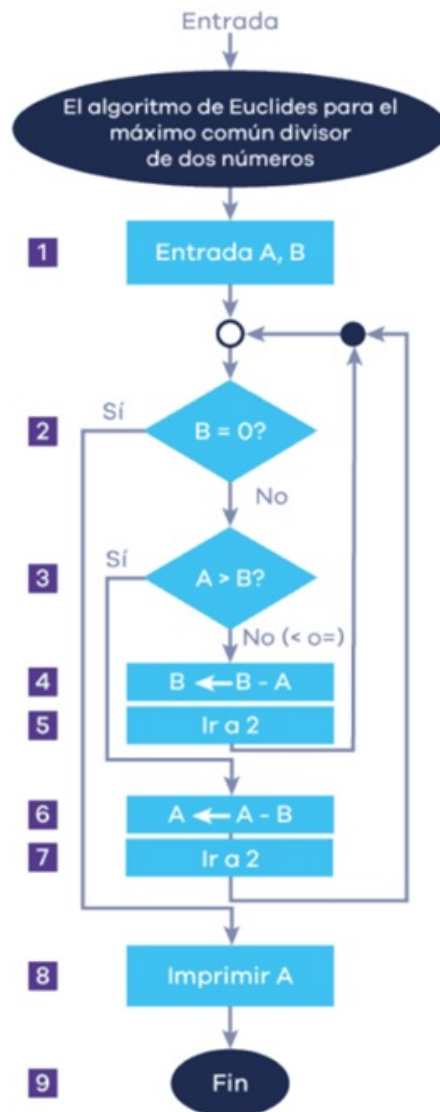


Ilustración 1. Diagrama de flujo ejemplo de la secuencia de pasos de un algoritmo

Los diagramas de flujo siguen unas **normas**, como el uso de rectángulos para las instrucciones y diamantes para las decisiones, utilizando flechas para conectar unos pasos con otros, etc.

Uno de los objetivos de los diagramas de flujo es ayudar a definir la entrada, el proceso y la salida de los algoritmos.

Tema 1. Programas informáticos y lenguajes de programación

Los diagramas de flujo ayudan a los programadores a **visualizar los pasos** de un algoritmo. Al igual que hacer un esquema antes de escribir un ensayo, los diagramas de flujo ayudan a organizar las ideas y usan símbolos específicos para representar diferentes partes de un algoritmo.

Los diagramas de flujo ayudan a **organizar las ideas** definiendo la entrada, el proceso y la salida de los algoritmos, a la vez que siguen unas normas, para las que usan símbolos específicos que representan las diferentes partes del algoritmo. El significado de los símbolos es el siguiente:

- ▶ Óvalo: comienzo o final del programa.
- ▶ Paralelogramo: operación de entrada.
- ▶ Rectángulo: instrucción o proceso a realizar.
- ▶ Diamante: decisión a tomar (el programa debe continuar por una de las dos rutas).
- ▶ Híbrido: operación de salida.
- ▶ Flechas: conectan unos pasos con otros e indican la dirección del flujo.

Tema 1. Programas informáticos y lenguajes de programación

	Óvalo	Comienzo o final del programa
	Paralelogramo	Operación de entrada
	Rectángulo	Instrucción o proceso a realizar
	Diamante	Decisión a tomar (el programa debe continuar por una de las dos rutas)
	Híbrido	Operación de salida
	Flechas	Conectan unos pasos con otros e indican la dirección del flujo

Notación algorítmica

Debe recoger las reglas de descripción de los objetos manipulados (informaciones de léxico), las operaciones puestas en juego (acciones del léxico) y el modo de organizar estas acciones en el tiempo (primitivas de control).

Acciones posibles:

- ▶ Acción elemental
- ▶ Composición secuencial
- ▶ Composición condicional.
- ▶ Composición alternativa.
- ▶ Composición selectiva.
- ▶ Composición iterativa.
- ▶ Acciones con nombres

Tema 1. Programas informáticos y lenguajes de programación

Notación para cada posible acción

Acción elemental

acción elemental

Composición secuencial

acción primera

acción segunda

...

acción n-ésima

Composición condicional

si condición entonces

acción si

fin si

Composición alternativa

si condición entonces

acción si

si no

acción no

fin si

Tema 1. Programas informáticos y lenguajes de programación

Composición selectiva

según indicador hacer

valor primero: acción primera

valor segundo: acción segunda

...

valor último: acción última

fin según

Composición iterativa

1. repetir ... hasta

repetir

acción

hasta condición.

2. mientras ... hacer

mientras condición hacer

acción

fin mientras

Tema 1. Programas informáticos y lenguajes de programación

3. para ...

para índice = valor inicial hasta índice = valor final hacer

acción

fin para

Acciones con nombre

acción nombre es

acciónprimera

acciónsegunda

...

acciónúltima

finacciónnombre

Partes de un algoritmo

La entrada de un algoritmo

Son los datos con los que el algoritmo opera. Ejemplos:

- ▶ Si se trata de una receta, la entrada serían los ingredientes.
- ▶ Si se trata de un algoritmo informático, la entrada sería, por ejemplo, un conjunto de datos.

Tema 1. Programas informáticos y lenguajes de programación

El proceso del algoritmo

Son los pasos y operaciones que sigue el algoritmo utilizando o no los datos de entrada:

- ▶ Si se trata de una receta, los diferentes pasos definidos y en orden que se han establecido
- ▶ Si se trata de un algoritmo informático, el proceso es la combinación ordenada de operaciones matemáticas, algebraicas, etc. que se han establecido por parte de los autores: sumas, restas, comparaciones, otros algoritmos, etc.

La salida del algoritmo

Es el resultado que entrega el algoritmo:

- ▶ Si se trata de una receta, la salida es el plato establecido por parte del autor.
- ▶ Si se trata de un algoritmo informático, la salida es el dato o conjunto de datos calculados esperados o un error

Los algoritmos recogen **operaciones** tan sencillas que pueden ser realizadas con éxito por cualquiera, incluso por máquinas: una máquina, ordenador, dispositivo, etc. puede ejecutar de forma automática y en tiempos ínfimos cualquier algoritmo sencillo (sumar, ordenar, repetir condiciones sencillas, etc.), y, por tanto, puede ejecutar cualquier algoritmo complejo siempre que pueda definirse como una combinación ordenada y concisa de algoritmos sencillos (divide y vencerás).

De esta manera las máquinas, ordenadores, etc., disponiendo de una mínima capacidad, pueden realizar casi cualquier operación por compleja que sea si nosotros la podemos **definir en términos** de otras **más simples**.

Tema 1. Programas informáticos y lenguajes de programación

El trabajo de los programadores informáticos, el cual consiste en traducir determinados problemas del mundo, entorno, ciencia o empresa a un lenguaje o lenguajes que una máquina pueda entender, es decir, en algoritmos que una máquina pueda ejecutar: para ello hay que describir una realidad en pequeños problemas simples en sucesión y poner al ordenador/dispositivo a la tarea

En tanto en cuanto se disponga del talento para la programación de algoritmos, así como la capacidad de procesamiento y la capacidad para guardar y disponer de datos, las posibilidades para resolver problemas se multiplican exponencialmente.

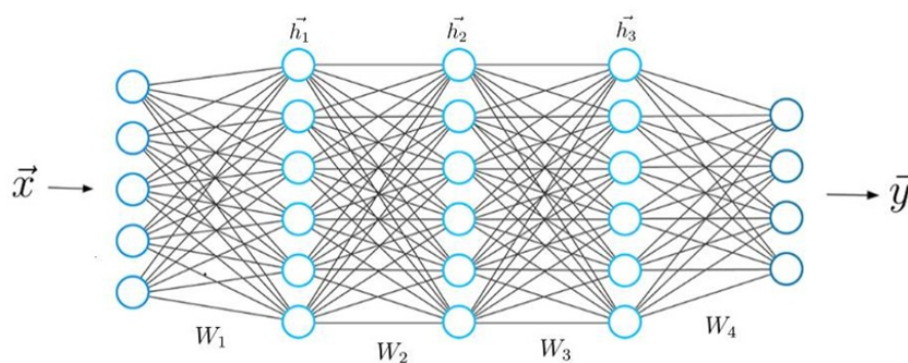


Ilustración 2. Ejemplo de red neuronal como algoritmo que combina cientos, miles o millones de neuronas, que recibiendo una entrada (datos en cualquier de sus formas) generan una salida.

Tal es la capacidad que la sociedad está alcanzando en las últimas décadas, tanto de desarrollar algoritmos complejos como de lograr capacidades asombrosas de procesamiento y almacenamiento de datos (arquitecturas de ordenadores en la nube, grandes capacidades de almacenamiento en bases de datos cada vez más veloces y un largo etc.), que realmente se consiguen resolver problemas verdaderamente retadores y ambiciosos.

Tema 1. Programas informáticos y lenguajes de programación

Problemas tan retadores y complejos que a veces es enormemente complicado poder describir lo que un algoritmo hace y cómo lo hace (interpretabilidad, explicabilidad) y si tienen un comportamiento ético.

Ejemplos de ello no dejan de surgir en los últimos tiempos en los ámbitos de las redes sociales y, en general, en cualquier ámbito donde implicamos evoluciones tecnológicas, entre ellos:

- ▶ Transporte (coches autónomos)
- ▶ Salud (genética, prevención, etc.)
- ▶ finanzas
- ▶ ciencia en general
- ▶ periodismo

Todo ello también genera el gran reto de evolucionar el marco regulatorio o legal para dar alcance al uso de los algoritmos para dar equilibrio a la utilidad que ofrecen y al coste o impacto que puede suponer su uso, privacidad, sesgos, etc.

Existen diferentes tipos de problemas: ordenar datos, buscar patrones, encontrar la ruta óptima, etc. A su vez, existen múltiples tipos de algoritmos para resolver problemas en diferentes campos de estudio: procesamiento de imágenes, criptografía, inteligencia artificial.

Cuando una solución se lleva a cabo mediante un algoritmo, la solución se vuelve **reutilizable**. Uno de los objetivos de la programación es el desarrollo de software reusable, que pueda ser reutilizado en multitud de proyectos reduciendo así los costes operativos.

Tema 1. Programas informáticos y lenguajes de programación

1.3. Pseudocódigo

El **pseudocódigo** es una versión simplificada de un código de programación en lenguaje plano utilizando frases cortas para escribir un código para un programa antes de ser implementado en un lenguaje de programación específico.

El pseudocódigo permite expresar **procesos computacionales**. Este tipo de código es cercano al ser humano (human-friendly) y no entendible por una máquina. Existen determinadas pautas sobre cómo escribir un pseudocódigo, pero está destinado a ser leído por humanos, no por una computadora.

El **pseudocódigo es formateado** para que cada línea represente una línea de código real en su programa final. El pseudocódigo no tiene por qué ser perfecto; solo debería dar una idea de cómo se verá el programa final.

Existen multitud de herramientas para asistir en la programación de pseudocódigo con fines docentes:

- ▶ PseInt
- ▶ PseudoEditor

PSeInt

PSeInt es una herramienta (libre, gratuita y multiplataforma) que permite escribir algoritmos de forma intuitiva mediante pseudolenguaje en español, cuyo objetivo es el aprendizaje de conceptos fundamentales, pero no el de los detalles del lenguaje de programación. Así, permite ejecutar y modificar el algoritmo (o parte del mismo) para observar su funcionamiento y verificar los resultados.

Tema 1. Programas informáticos y lenguajes de programación

Cabe destacar que detecta errores y ofrece sugerencias para corregirlos. Además, PSeInt permite trabajar con diagramas de flujo, convirtiendo automáticamente los algoritmos entre una y otra representación, así como editarlos en cualquiera de los formatos.

<https://sourceforge.net/projects/pseint/>

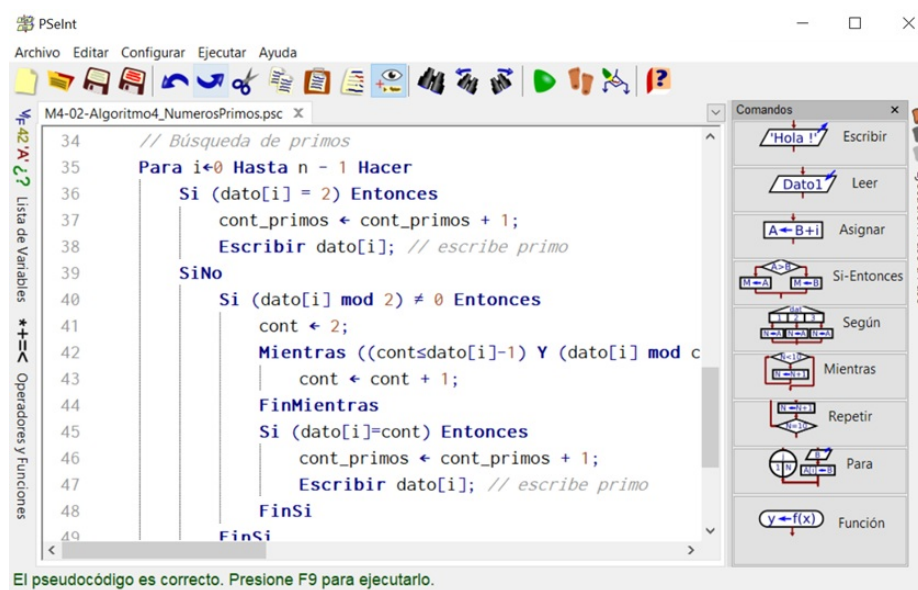


Ilustración 3. Ejemplo de algoritmo para la búsqueda de números primos realizado con pseudocódigo sobre la herramienta PSeInt.

PseudoEditor

PseudoEditor es un editor de pseudocódigo online en inglés y gratuito. Entre sus características se encuentra el resaltado de sintaxis dinámica para palabras clave, funciones, tipos de datos, etc., lo que ayuda a escribir y depurar el pseudocódigo de forma rápida.

Adicionalmente, permite el almacenamiento en la nube y destaca los errores, entre otras funciones.

Tema 1. Programas informáticos y lenguajes de programación

1.4. Programas informáticos

Un **programa** es el código escrito para resolver un problema siguiendo las reglas sintácticas de un lenguaje de programación.

Un **programa de ordenador** es un conjunto de sentencias escrito en un lenguaje de programación que acaba convertido en decenas, cientos o miles de sencillas operaciones que se realizan en base a señales eléctricas manejadas por los circuitos electrónicos de los dispositivos (procesadores y otros).

Los programas dicen a las máquinas lo que deben hacer. Un programa puede ser desde lo más sencillo, por ejemplo, un programa que imprima el mensaje “Bienvenidos” hasta lo más complejo, un navegador, un sistema operativo, etc.

El conjunto de programas informáticos y los datos utilizados por los mismos se conoce como **software**.

Mientras que los algoritmos pueden ser aplicados en el entorno físico en la vida real (una receta de cocina) los **programas informáticos** precisan de una **máquina** para ser ejecutados. Un programa puede contener o no algoritmos y llevar a cabo más tareas, como por ejemplo comunicarse con otro programa, reaccionar ante eventos, recibir información del usuario, etc.

Los programas informáticos consumen **recursos computacionales** del sistema informático en el que se ejecutan: memoria RAM, procesamiento, espacio en disco, etc. Es por tanto que una máquina puede ejecutar un número limitado de programas al mismo tiempo.

Tema 1. Programas informáticos y lenguajes de programación

Un programa informático puede ser ejecutado en diferentes **tipos de dispositivos**, algunos de ellos:

- ▶ Un ordenador personal
- ▶ Una estación de trabajo
- ▶ Un servidor
- ▶ Un teléfono inteligente
- ▶ Un dispositivo IoT (Internet de las cosas)

Tema 1. Programas informáticos y lenguajes de programación

1.5. Lenguajes de programación

Los **lenguajes de programación** expresan cálculos de una forma comprensible tanto para personas como para máquinas, permiten dar instrucciones a éstas en un lenguaje que entienden, además de procesar de forma rápida y eficiente grandes y complejas cantidades de información.

Todos los **lenguajes de programación** que se pueden utilizar para comunicarse con las máquinas son reducidos al lenguaje más básico de todos, el único que las máquinas entienden: el lenguaje "**binario**" o "código máquina", el cual es la reducción de cualquier lenguaje a secuencias de ceros y unos.

Este **código binario** o **máquina** recibe su nombre por cómo las máquinas procesan la información, dado que lo hacen mediante circuitos eléctricos que pueden encenderse y apagarse. Se usan los dígitos 1 y 0 para representar encendido y apagado, respectivamente.

La **CPU** de un ordenador solo puede leer instrucciones escritas en binario, por lo que cuando una persona introduce un programa en una computadora, otro programa lo convierte en código binario. A este proceso de traducción de un programa a código máquina se le llama "**compilación**".

Existen cientos de lenguajes de programación de uso generalizado, cada uno con sus propias complejidades e idiosincrasias.

Dependiendo de los fines e intereses del proyecto de software, un equipo tecnológico elegirá uno u otro, dependiendo de si va a desarrollar una página web, un videojuego, una aplicación, etc.

Tema 1. Programas informáticos y lenguajes de programación

Todos los años el **IEEE** (Institute of Electrical and Electronics Engineers) elabora un ranking de **lenguajes de programación de mayor relevancia** en la industria del software.

En el ranking, cada lenguaje de programación recibe una puntuación de 0 a 100 según la demanda y el número de consultas de búsqueda y menciones de este, siendo el lenguaje de programación con la puntuación más alta el que ocupa el primer lugar. En el año 2021 el ranking ha sido el siguiente:

Rank	Language	Type	Score
1	Python	  	100.0
2	Java	  	95.4
3	C	  	94.7
4	C++	  	92.4
5	JavaScript		88.1
6	C#	   	82.4
7	R		81.7
8	Go	 	77.7
9	HTML		75.4
10	Swift	 	70.4

Ilustración 4. Ranking de lenguajes de programación según IEEE. <https://spectrum.ieee.org/top-programming-languages/#toggle-gdpr>

Tema 1. Programas informáticos y lenguajes de programación

De la misma manera, el índice TIOBE muestra un listado actualizado mensualmente de los **lenguajes de programación más populares** (aquellos para los cuales se han escrito más líneas de código). A continuación, una lista de los cinco más populares en octubre de 2021:

Oct 2021	Oct 2020	Change	Programming Language	Ratings	Change
1	3	▲	 Python	11.27%	-0.00%
2	1	▼	 C	11.16%	-5.79%
3	2	▼	 Java	10.46%	-2.11%
4	4		 C++	7.50%	+0.57%
5	5		 C#	5.26%	+1.10%

Ilustración 5. Ranking de lenguajes de programación según TIOBE. <https://www.tiobe.com/tiobe-index/>

Por otra parte, la encuesta anual de JetBrains ‘El estado de ecosistema de desarrollo’ muestra los **lenguajes de programación más utilizados**, así como frameworks y herramientas relacionadas con cada uno. También comparten conclusiones clave derivadas del análisis de los lenguajes y la evolución de los mismos.

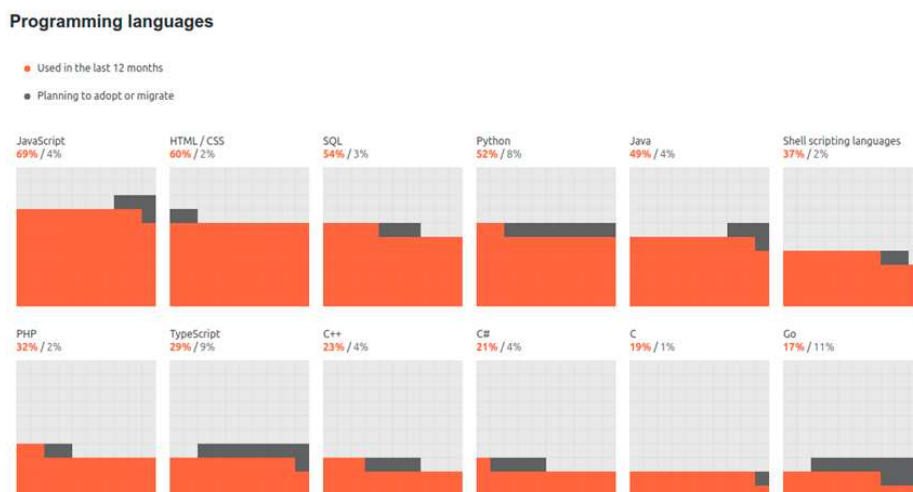


Ilustración 6. Ranking de lenguajes de programación según JetBrains.

<https://www.jetbrains.com/lp/devecosystem-2021/>

Tema 1. Programas informáticos y lenguajes de programación

Pensamiento computacional

El **pensamiento computacional** es el proceso de pensamiento para crear soluciones que pueden ser llevadas a cabo por una computadora.

Un ejemplo podría ser desarrollar un programa de inteligencia artificial (IA) que pueda comprender el lenguaje natural (la forma en que los humanos hablan entre ellos), analizar un libro y escribir un ensayo sobre el mismo.

En computación, la inteligencia artificial (IA) es una forma eficaz de programar máquinas para que puedan aprender de grandes cantidades de datos para mejorar la forma en que funcionan. Este tipo de computación permite que los programas puedan hacer predicciones y tomar decisiones más allá de aquellas para las que están programados directamente.

El pensamiento computacional se compone de diferentes elementos (descomposición, reconocimiento de patrones, abstracción, diseño de algoritmos, etc.) que se detallan a continuación.

Descomposición

La **descomposición** consiste en dividir un problema en partes más sencillas para que los problemas sean más manejables y fáciles de resolver.

Para poder encontrar una solución satisfactoria a un problema, en primer lugar, se deben comprender todas las partes del mismo.

Una vez se comprenda cada parte, se podrá dividir el problema en tareas más simples (descomposición).

Tema 1. Programas informáticos y lenguajes de programación

Reconocimiento de patrones

Identificar elementos comunes, es decir, lo que tienen en común distintos problemas.

Los patrones son eventos que se repiten. Reconocer dónde se han creado soluciones para problemas similares antes será útil a la hora de crear soluciones que puedan usarse para completar diferentes tareas.

Abstracción

Separar los detalles que importan de los que no.

La abstracción se centra en las ideas importantes de un problema e ignora los detalles que no son de ayuda a la hora de encontrar una solución.

De esta forma la abstracción reduce la complejidad por medio de ocultar detalles completos en diferentes niveles de entendimiento.

Diseño de algoritmos

Crear una solución con pasos simples que cualquiera pueda seguir.

El **diseño de algoritmos** consiste en escribir los pasos que se deben seguir para poder obtener la misma solución en todo momento.

Cuando una solución se lleva a cabo mediante un algoritmo, la solución se vuelve reutilizable.

Tipos de programas

Los **programas informáticos** pueden ser clasificados en diferentes categorías en función del objetivo para el que son desarrollados:

- **Software de sistema:** programas que permiten la interacción entre el usuario y el hardware. Constituyen la plataforma o entorno sobre el que los demás tipos de software pueden ejecutarse.

Tema 1. Programas informáticos y lenguajes de programación

- ▶ **Software de aplicación:** programas de usuario final, utilizados en el entorno empresarial, educativo, recreativo, etc. Constituyen las aplicaciones software utilizadas hoy en día en ordenadores, teléfonos inteligentes, etc.
- ▶ **Software de programación:** programas para la asistencia en el desarrollo de software. Comúnmente conocidos como entornos de desarrollo integrado o por sus siglas en inglés IDEs.

Software de sistema

Algunos ejemplos de **software de sistema** pueden ser:

- ▶ Sistemas operativos
- ▶ Drivers de dispositivos
- ▶ Firmware
- ▶ Traductores de lenguajes de programación
- ▶ Utilidades

Software de aplicación

Algunos ejemplos de **software de aplicación** pueden ser:

- ▶ Procesadores de texto
- ▶ Hojas de cálculo
- ▶ Navegadores web
- ▶ Software multimedia

Tema 1. Programas informáticos y lenguajes de programación

- ▶ Software de bases de datos
- ▶ Aplicación móvil
- ▶ CRM

Software de programación

Algunos ejemplos de **software de programación** pueden ser:

- ▶ Herramientas para programar (**IDEs**): Eclipse, herramientas de JetBrains, Visual Studio, etc.
- ▶ Herramientas de acceso a **bases de datos**: DataGrip, DBeaver, pgAdmin, MySQL Workbench, etc.
- ▶ Herramientas de **testing**: Katalon Studio, Postman.

Tema 2. Características de los lenguajes de programación para IA

2.1. Introducción y objetivos

Para estudiar este tema lee las Ideas Clave que se presentan a continuación y revisa los contenidos adicionales para mejorar la comprensión y aumentar los conocimientos sobre las materias tratadas.

Este tema supone un recorrido por los fundamentos de los lenguajes de programación, los diferentes paradigmas y elementos que los caracterizan. Los objetivos más concretos son:

- ▶ Describir la **evolución de los lenguajes de programación** desde el lenguaje máquina a los lenguajes de alto nivel.
- ▶ Conocer los **componentes comunes** a todos los lenguajes de programación.
- ▶ Entender los distintos **paradigmas de programación** y sus diferencias.
- ▶ Saber **elegir un paradigma de programación** en función de las necesidades.
- ▶ Distinguir las características que hacen que un lenguaje de programación sea idóneo para el desarrollo de **inteligencia artificial**.

Tema 2. Características de los lenguajes de programación para IA

2.2. Evolución de los lenguajes de programación

Para escribir programas informáticos es necesario el uso de lenguajes de programación. Los lenguajes de programación son un conjunto predefinido de palabras que se combinan de acuerdo con una serie de reglas predefinidas (sintaxis).

Los primeros lenguajes de programación son conocidos como **lenguajes máquina**, compuestos por **código binario**. Este tipo de lenguajes es el único que entienden las máquinas y supone mucha dificultad para los programadores desarrollar tareas complejas.

Es por esta razón que los lenguajes de programación han ido evolucionando década tras década para acercarse más al lenguaje natural de las personas, con el objetivo de facilitar la programación de tareas complejas a los programadores.

De esta forma, los lenguajes de programación han evolucionado de la siguiente manera:

- ▶ Lenguajes máquina
- ▶ Lenguajes ensambladores
- ▶ Lenguajes de alto nivel

Lenguaje máquina

La primera generación de lenguajes de programación son los lenguajes máquina. El **lenguaje máquina** recibe su nombre por ser el único que entiende el hardware de las máquinas.

El hardware de las máquinas se compone de circuitos formados por interruptores eléctricos con dos estados: **encendido** (equivale a 1) y **apagado** (equivale a 0).

Tema 2. Características de los lenguajes de programación para IA

Debido a que solo hay dos estados posibles, apagado o encendido (0 o 1), estos lenguajes reciben el nombre de **código binario**.

El principal problema del lenguaje máquina es su **dependencia con el hardware**, lo que quiere decir que el lenguaje máquina utilizado para un hardware concreto tendrá que ser distinto para una máquina que utilice un hardware diferente.

Este inconveniente sumado a la **dificultad de programar y encontrar errores** en este tipo de lenguajes hace que no sean idóneos para el día a día de la programación de aplicaciones informáticas.

Lenguaje ensamblador

Para mitigar los inconvenientes del lenguaje máquina, surge una **segunda generación** de lenguajes de programación conocidos como lenguajes ensambladores.

La idea principal de los **lenguajes ensambladores** es la de reemplazar el código binario en la fase de programación con el fin de facilitar su desarrollo.

El código ensamblador está compuesto por símbolos, mnemónicos que simbolizan instrucciones, secciones de datos y directivas del ensamblador.

Como el hardware únicamente entiende el código máquina es necesario el uso de una herramienta (**ensamblador**) que traduzca el código ensamblador al código máquina.

El principal inconveniente es que los lenguajes ensambladores siguen siendo específicos de hardware sobre el que se ejecutan, es decir, cada arquitectura de procesador tiene su propio lenguaje ensamblador específico.

Tema 2. Características de los lenguajes de programación para IA

Lenguajes de alto nivel

Para mitigar los inconvenientes de los lenguajes ensambladores surge una tercera generación de lenguajes conocida como los **lenguajes de alto nivel**.

El principal objetivo de los lenguajes de alto nivel es proporcionar **portabilidad** al permitir desarrollar programas independientes de la máquina sobre la que se ejecutan.

Otro de los principales objetivos es proporcionar una sintaxis similar al **lenguaje natural** con el fin de facilitar el proceso de desarrollo de nuevos programas. Es por esta razón que estos lenguajes se consideran de alto nivel, por estar más cerca del lenguaje humano que del lenguaje de las máquinas.

Los lenguajes de programación han seguido evolucionando eventualmente en una **cuarta y quinta generación** proporcionando características para facilitar aún más la creación de nuevos programas.

Debido a esto, muchos lenguajes que inicialmente eran considerados de alto nivel ahora son considerados de bajo nivel. Ejemplos de **lenguajes de alto nivel** son: Java, JavaScript, Python, Ruby, C++, PHP, etc.

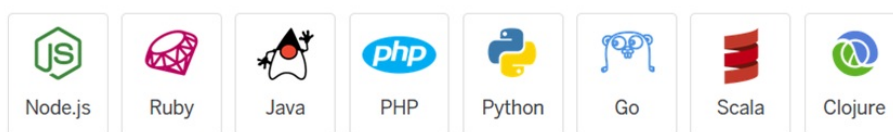


Ilustración 1. Ejemplos de lenguajes de programación de alto nivel. Fuente: <https://www.heroku.com/>

Al igual que ocurre con los lenguajes ensambladores, los lenguajes de alto nivel deben ser **traducidos a lenguaje máquina** para poder ser ejecutados sobre el hardware. En función de cómo se realiza esa traducción, los lenguajes de alto nivel pueden ser **compilados** o **interpretados**.

Tema 2. Características de los lenguajes de programación para IA

En función de cómo es leído el código programado por la máquina destino, un lenguaje puede ser **compilado** o **interpretado**.

En los **lenguajes compilados** el compilador traduce el código de alto nivel (código fuente) al lenguaje máquina.

En los lenguajes **interpretados**, un software llamado intérprete es el que lee y ejecuta el código en la máquina destino línea a línea.

Compiladores

Los **compiladores** son programas informáticos que traducen el código fuente desarrollado en un lenguaje de alto nivel hacia código máquina, creando así programas ejecutables.

El compilador lleva a cabo la **traducción de un lenguaje a otro** por medio de una serie de fases como el preprocesado, análisis léxico, análisis sintáctico, análisis semántico, etc.

Intérpretes

Un **intérprete** es un programa informático que analiza el código fuente y lo traduce a código máquina instrucción por instrucción cuando se ejecuta.

La principal desventaja de los intérpretes es que la interpretación del código es más lenta que la ejecución del código compilado ya que el intérprete debe analizar cada sentencia del programa y ejecutarla. Es por esta razón que se les asocia un mejor rendimiento a los lenguajes compilados.

Tema 2. Características de los lenguajes de programación para IA

Por otro lado, el tiempo tomado para compilar un programa puede ser más elevado que el tiempo usado por los intérpretes en analizar y ejecutar. Por tanto, los intérpretes pueden ofrecer una **mayor flexibilidad en las fases de desarrollo** al evitar tener que esperar que todo el código se compile para poder ejecutarlo y probarlo.

En general, los **lenguajes de programación de alto nivel** tienden a asociarse con una de estas dos categorías, lenguajes **compilados** o **interpretados**. En la práctica también existen lenguajes que emplean ambos procesos, teniendo una fase de compilación a código intermedio o *bytecode* y una fase de interpretación de este *bytecode*. Es el caso de lenguajes como Java o Lisp.

Tema 2. Características de los lenguajes de programación para IA

2.3. Sintaxis y estructura de los lenguajes de programación

Todos los lenguajes de programación comparten elementos comunes, aunque cada uno tiene su propia sintaxis para expresarlos:

- ▶ Tipos de datos
- ▶ Variables
- ▶ Estructuras de datos
- ▶ Operadores
- ▶ Estructuras de control
 - Condicionales
 - Repetitivas
- ▶ Funciones

Tipos de datos

Los programas informáticos trabajan con datos. Esos datos pueden ser de diferente naturaleza o tipo: números, textos, caracteres, valores booleanos, etc.

Un **sistema de tipos** permite clasificar los valores y expresiones en diferentes tipos de datos y las operaciones permitidas con cada uno de ellos. El objetivo de este sistema es verificar la validez de las sentencias programadas cuando se utilizan datos, establece las formas en las que los programas pueden ser escritos.

Tema 2. Características de los lenguajes de programación para IA

Tipado fuerte vs. Tipado débil

Un lenguaje de programación **débilmente tipado** permite que un valor de un tipo pueda ser operado como un valor de otro tipo, por ejemplo, intercambiar textos y números. Este mecanismo proporciona flexibilidad al poder trabajar con un mismo dato en diferentes contextos.

Un lenguaje de programación **fuertemente tipado** no permite realizar operaciones sobre tipos equivocados. Este mecanismo proporciona robustez y seguridad al evitar posibles errores derivados de operaciones con distintos tipos.

Tipado estático vs. Tipado dinámico

Un lenguaje de programación tiene **tipado estático** cuando las comprobaciones de tipos se realizan durante la compilación, de forma que si alguna se incumple el código no puede compilarse.

Por el contrario, un lenguaje de programación es **dinámicamente tipado** si la comprobación de tipos se realiza durante la ejecución del programa en lugar de su compilación.

Variables

Una **variable** en programación es un contenedor de información, contiene un dato en memoria durante la ejecución del programa. Las variables permiten reutilizar un valor en memoria a lo largo de diferentes partes de un programa.

Las variables se conforman con un **identificador** y un **valor**. El identificador es el nombre que el programador asigna con el objetivo de hacer referencia a la misma en diferentes puntos del programa.

El valor es el dato que almacenan. Puede ser desde un valor numérico entero, decimal hasta caracteres o texto.

Tema 2. Características de los lenguajes de programación para IA

```
variable1 = 3
variable2 = 'Hola'
variable3 = 'Mundo'
variable4 = 5.7
variable5 = 5 + 6j
variable6 = True
```

Ilustración 2. Ejemplo de variables en Python.

Creación de variables

Una **variable** puede ser **declarada**, lo que significa que únicamente se crea con un identificador, pero no se le asigna un valor.

Por medio de un **operador de asignación**, una variable previamente declarada puede tomar un valor.

También es posible inicializar una variable al mismo tiempo que es declarada, en una misma línea de código.

Nomenclatura

La **nomenclatura** utilizada para crear el identificador de una variable es elegida por el programador, pero debe seguir una serie de reglas y buenas prácticas:

- ▶ No debe contener espacios.
- ▶ No debe comenzar por un número.
- ▶ No puede ser una palabra reservada del lenguaje de programación que se esté utilizando.

Tema 2. Características de los lenguajes de programación para IA

- ▶ Debería comenzar por letra minúscula.
- ▶ Sigue unas convenciones de formato, por ejemplo:
 - En Java se utiliza el formato `camelCase`.
 - En Python se utiliza el formato `snake_case`.

Estructuras de datos

Las variables permiten almacenar un solo dato a la vez. En ocasiones es necesario trabajar con más de un dato al mismo tiempo, por ejemplo, una lista de emails de contacto.

Las estructuras de datos son contenedores específicos para trabajar con más de un dato simultáneamente.

```
# listas
names = ['Alberto', 'Mary', 'Mike']
# tuplas
customer = (1, 'Larry', 34, 50000, True)
# diccionario
employee = {
    "customer_id": 1,
    "store_id": 1,
    "first_name": "MARY",
    "last_name": "SMITH",
}
# conjunto
laptops = {"asus", "dell", "hp", "msi", "lenovo"}
```

Ilustración 3. Ejemplo de estructuras de datos en Python: listas, tuplas, diccionarios y conjuntos.

Tema 2. Características de los lenguajes de programación para IA

Operadores

Los **operadores** son símbolos que indican al compilador o intérprete la ejecución de una determinada operación aritmética, lógica o de comparación.

Los operadores permiten llevar a cabo cálculos y evaluar condiciones haciendo uso de variables.

En cada lenguaje se pueden definir de una manera, ya sea mediante un símbolo o con letras.

```
# Operadores aritméticos
x = 5
y = 7
# suma
result = x + y
# resta
result = x - y
# multiplicación
result = x * y
# division
result = x / y
# modulo (resto de la division)
result = x % y
# exponente
result = x ** y
# division floor o division suelo
result = x // y
```

Ilustración 4. Ejemplo de operadores aritméticos en Python.

Tema 2. Características de los lenguajes de programación para IA

Operadores aritméticos

Los **operadores aritméticos** permiten llevar a cabo operaciones matemáticas sobre datos numéricos. Algunos ejemplos extraídos de diferentes lenguajes de programación:

- ▶ Suma: +
- ▶ Resta: -
- ▶ Multiplicación: *
- ▶ Exponenciación: **
- ▶ División: /
- ▶ División suelo: //
- ▶ Resto: %

Operadores de comparación

Operadores de comparación, el resultado es verdadero o falso:

- ▶ Mayor que: >, >=
- ▶ Menor que: <, <=
- ▶ Igual que: =
- ▶ Igual que con comprobación de tipos: ===
- ▶ Distinto que: !=

Tema 2. Características de los lenguajes de programación para IA

Operadores lógicos

Permiten evaluar más de una condición a la vez:

- ▶ Conjunción: y (and)
- ▶ Disyunción: o (or)
- ▶ Negación: ! (not)

Operadores de asignación

Permiten asignar un valor a una variable además de poder realizar una operación aritmética a la vez y asignar el resultado:

- ▶ =
- ▶ +=
- ▶ -=
- ▶ *=
- ▶ /=

Estructuras de control

El flujo de ejecución normal de los programas es **secuencial**, de arriba abajo, sentencia a sentencia.

Ese flujo de ejecución puede ser alterado utilizando **estructuras de control** que permitan crear bifurcaciones en el código además de poder repetir determinados fragmentos.

Para ello existen las estructuras de control **condicional** y de **repetición**.

Tema 2. Características de los lenguajes de programación para IA

El principal objetivo de las estructuras de control es poder crear **programas más flexibles** y con **mayor rango de posibles acciones** al permitir evaluar distintas condiciones además de iterar sobre estructuras de datos.

Estructuras de control condicional

Las **estructuras de control condicional** permiten evaluar una o más condiciones a fin de ejecutar un bloque de código y u otro en función de si se cumplen o no.

Las estructuras condicionales más comunes entre los diferentes lenguajes de programación son:

- ▶ **If:** equivale a Si. Ejecuta el código que envuelve si se cumple la condición o condiciones que evalúa.
- ▶ **Else:** equivale a Entonces. Ejecuta el código que envuelve si no se cumple la condición o condiciones que evalúa la estructura if anterior.
- ▶ **Else-If:** equivale a Entonces Si. Ejecuta el código que envuelve si no se cumple la condición o condiciones que evalúa la estructura if anterior y además se cumple las que envuelve con su propio if.
- ▶ **Switch:** equivale a un interruptor. Funciona de manera similar a un if combinado con else-if. Evalúa los diferentes casos posibles para el valor de una variable y proporciona un código para cada uno de ellos.

Tema 2. Características de los lenguajes de programación para IA

```
if check1:
    print("")
elif check2:
    print("")
elif check3:
    print("")
elif check4:
    print("")
else:
    print("else")
```

Ilustración 5. Ejemplo de estructuras de control en Python.

Dentro de las estructuras condicionales se encuentran a menudo operadores lógicos and y or que permiten evaluar más de una condición a la vez.

```
if check1 or check2:
    print("Verdadero")
    print("Verdadero2")

if check1 or (check2 and check3):
    print("Verdadero")
    print("Verdadero2")
```

Ilustración 6. Ejemplo de estructuras de control en Python utilizando operadores lógicos.

Tema 2. Características de los lenguajes de programación para IA

Estructuras de control repetitivo

Las **estructuras de control repetitivo** permiten que un mismo bloque de código pueda ser ejecutado más de una vez durante la ejecución de un programa.

Se utilizan principalmente cuando se quiere realizar una operación varias veces, por ejemplo, recorrer los números en una estructura de datos para calcular el sumatorio.

Las estructuras de control repetitivo pueden ser **determinadas** o **indeterminadas**.

Una estructura de control repetitivo es determinada cuando tiene acotado un rango máximo de repeticiones, es el caso de los **bucles for**.

```
text = "tesla"

for char in text:
    print(char)
```

Ilustración 7. Ejemplo de estructura de repetición for en Python.

Por otro lado, las estructuras de control repetitivo son **indeterminadas** cuando la repetición depende de si se cumplen ciertas condiciones. Las condiciones se verán alteradas dentro del bucle de forma que cuando se logre el objetivo las condiciones terminen la ejecución de este. Es el ejemplo de los **bucles while**.

Tema 2. Características de los lenguajes de programación para IA

```
paths = pathname.split('/')
while '.' in paths:
    paths.remove('.')
if not paths:
    return os.curdir
return os.path.join(*paths)
```

Ilustración 8. Ejemplo de estructura de repetición while en Python.

Funciones o procedimientos

Una **función** o procedimiento es un bloque de código que puede ser reutilizado en diferentes partes de un programa sin necesidad de duplicar el código que contiene.

Las funciones tienen dos partes bien diferenciadas:

- ▶ La **signatura**: es el conjunto de palabras que identifican la función: su visibilidad, su tipo de retorno, su identificador, sus parámetros.
- ▶ El **cuerpo**: de la función es el código que ejecuta, puede ser una o múltiples instrucciones.

Las variables y estructuras de datos que se crean dentro de una función existen durante la ejecución de esta, pero no fuera.

Tema 2. Características de los lenguajes de programación para IA

```
def hola3(name='Ricardo'):  
    print('Hola {}'.format(name))  
  
hola3()
```

Ilustración 9. Ejemplo de funciones en Python.

Tema 2. Características de los lenguajes de programación para IA

2.4. Paradigmas de programación

La complejidad es el principal factor por el cual el código de los programas se vuelve complejo y difícil de mantener en el tiempo. Cuanta menos complejidad tenga un programa, más fácil será de leer y también de mantener y evolucionar, por ende, menos costes implicará su desarrollo.

Uno de los principales enfoques para lidiar con la complejidad son los **paradigmas de la programación**.

Un paradigma de programación hace referencia a un estilo de programación. No se trata de un lenguaje específico, si no de la forma en la que se programa y estructura el código.

Existen dos principales familias de **paradigmas de programación**:

- ▶ Programación **imperativa**
- ▶ Programación **declarativa**

Programación imperativa

La **programación imperativa** es un paradigma a través del cual se especifican en el código todos los pasos que debe realizar el programa para lograr un objetivo. Para ello se utilizan variables, estructuras de control condicional y repetitivo.

La **programación imperativa** se encarga de describir el **cómo** se debe llevar a cabo un proceso paso por paso, definiendo el flujo de control como sentencias que cambian el estado del programa.

Tema 2. Características de los lenguajes de programación para IA

Dentro de la programación imperativa existen múltiples enfoques:

- ▶ Programación **estructurada**
- ▶ Programación **orientada a objetos**

Programación estructurada

La **programación estructurada** es un paradigma que busca mejorar la legibilidad y reducir la complejidad de un código por medio del uso de **sentencias de control**.

La programación estructurada emplea 3 estructuras para la creación de programas:

- ▶ Secuencia. La estructura secuencial es la que se da de forma natural en el lenguaje, porque las sentencias se ejecutan en el orden en el que aparecen en el programa, es decir, una detrás de la otra.

Secuencia

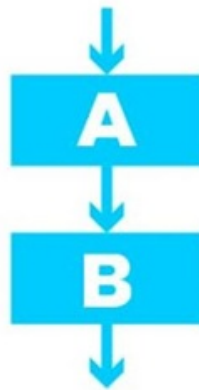


Ilustración 10. Ejemplo de estructura secuencial.

Tema 2. Características de los lenguajes de programación para IA

- **Selección o condicional.** La estructura condicional se basa en que una sentencia se ejecuta según el valor que se le atribuye a una variable booleana. Una variable booleana es aquella que tiene dos valores posibles. Por tanto, esta estructura se puede ejecutar de dos formas distintas, dependiendo del valor que tenga su variable.

Selección o condicional

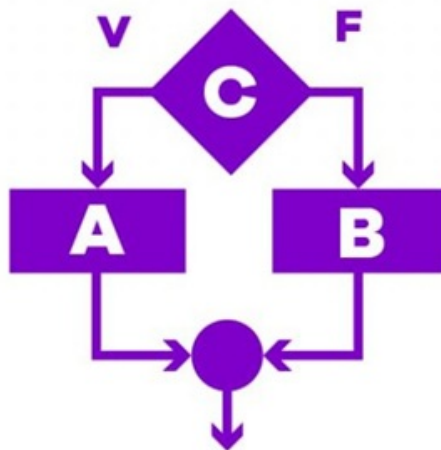


Ilustración 11. Ejemplo de estructura de control condicional.

- **Iteración (ciclo o bucle).** La estructura de repetición ejecuta una o un conjunto de sentencias siempre que una variable booleana sea verdadera. Para los bucles o iteraciones, los lenguajes de programación usan las estructuras while y for.

Tema 2. Características de los lenguajes de programación para IA

Iteración (ciclo o bucle)



Ilustración 12. Ejemplo de estructura de control repetitivo o de iteración.

La **programación procedimental** es un tipo de programación estructurada y se basa en el uso de procedimientos o funciones que modifican el estado del programa.

Programación orientada a objetos

La **Programación Orientada a Objetos (POO)** es un paradigma que organiza los programas como objetos, elementos compuestos por variables (estado) y métodos (comportamiento).

Los elementos básicos de la programación orientada a objetos son las **clases** y los **objetos**. Las clases actúan como plantillas que especifican los atributos y métodos que tendrán los objetos.

Tema 2. Características de los lenguajes de programación para IA

Los **objetos** son las **instancias** particulares que se crean a partir de una clase. Ejemplo: a partir de la clase vehículo se pueden crear muchos objetos vehículo, uno para representar un Toyota, otro para representar un Ford, uno puede estar en marcha, otro estacionado, etc.

Otros **mecanismos** de la programación orientada a objetos son:

- ▶ Encapsulación
- ▶ Herencia
- ▶ Abstracción
- ▶ Polimorfismo
- ▶ Serialización
- ▶ Asociación
- ▶ Composición

La programación orientada a objetos se utiliza especialmente en el desarrollo de programas complejos y grandes **sistemas de software** de forma **escalable**.

Dentro de la programación orientada a objetos se emplea el uso de patrones de diseño con el fin de promover el uso de **buenas prácticas** y la creación de código de calidad.

Los **patrones de diseño** son soluciones a problemas comunes en software, no son código en sí, son formas de estructurar los elementos de la programación orientada a objetos (clases, objetos, abstracción).

Tema 2. Características de los lenguajes de programación para IA

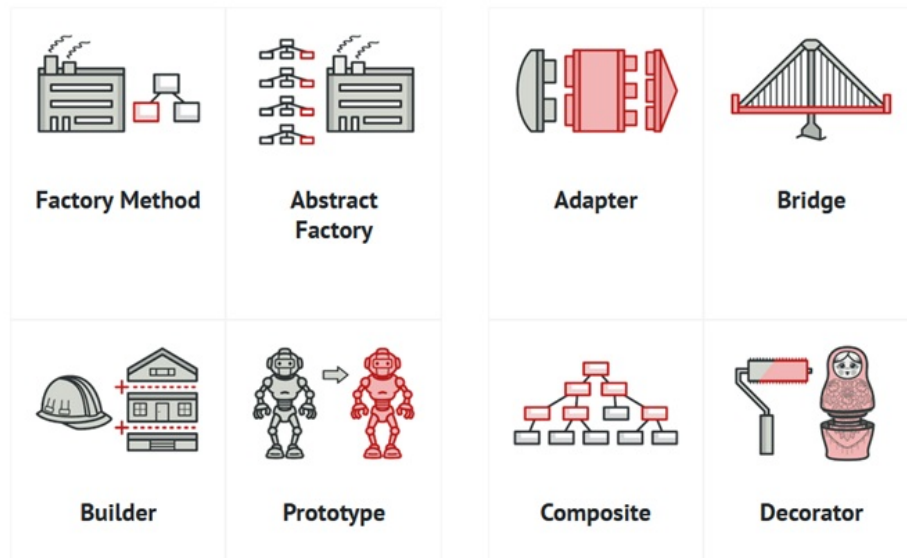


Ilustración 13. Ejemplo de estructuras de datos en Python. Fuente: <https://refactoring.guru/design-patterns/catalog>

Programación declarativa

De forma paralela a la programación imperativa se desarrolló el paradigma declarativo. La **programación declarativa** define la lógica del programa, pero no detalla el flujo de control.

En la **programación declarativa** el programador define qué es lo que espera del programa sin definir cómo debe ser implementado, es decir, sin programar los pasos necesarios para obtenerlo. Este paradigma se enfoca en qué es necesario obtener en lugar de especificar el cómo obtenerlo.

Dentro del paradigma de la programación declarativa destacan algunos subparadigmas como:

- ▶ Programación funcional
- ▶ Programación lógica

Tema 2. Características de los lenguajes de programación para IA

Programación funcional

Entre los elementos sintácticos de los lenguajes de programación se encuentran las funciones. Una función es un bloque de sentencias para ejecutar una determinada tarea de forma reutilizable a lo largo de un código.

La **programación funcional** es un paradigma que consta de llamadas a funciones concatenadas en las que cada parte del programa se interpreta como una función.

De esta forma, dentro de la programación funcional las funciones pueden adoptar distintas estructuras.

Un ejemplo de uso de la programación funcional es la parametrización del comportamiento, la cual consiste en poder utilizar como parámetro de una función otra función. Siendo posible también utilizar una función como resultado o retorno de otra función.

La **programación funcional** se emplea a menudo en conjunto a la programación orientada a objetos, permitiendo la creación de aplicaciones flexibles y escalables.

Características de la programación funcional:

- ▶ Inmutabilidad de los datos
- ▶ Transparencia referencial
- ▶ Modularidad
- ▶ Mantenibilidad
- ▶ Funciones puras
- ▶ Inmutabilidad de los datos

Tema 2. Características de los lenguajes de programación para IA

Se debe poder crear nuevas variables y estructuras de datos **sin tener que modificar** las ya existentes.

Al evitar los datos existentes y crear copias se evitan posibles **efectos secundarios**.

Favorece el uso del **paralelismo**, evitando que una ejecución altere el estado de otra ejecución que tiene lugar simultáneamente.

- ▶ Transparencia referencial

Una función es **referencialmente transparente** si siempre devuelve el mismo resultado cuando es invocada con los mismos valores de entrada en sus argumentos.

Siempre devuelve el mismo resultado para la misma entrada, independientemente de que se ejecute múltiples veces y en diferentes lugares de un programa.

- ▶ Modularidad

La **modularidad** es una técnica para diseñar software en pequeños módulos o fragmentos independientes, cada uno con todo lo necesario para ser ejecutado y proporcionar una **funcionalidad**.

La modularidad incrementa la **productividad** debido a que los módulos pueden ser programados más rápido y con capacidad para ser reutilizados en diferentes programas.

Alternativamente, los programas modulares pueden ser **probados** y **depurados** de manera más sencilla y rápida.

- ▶ Mantenibilidad

La **mantenibilidad** de un software hace referencia a la facilidad con la que se puede evolucionar y corregir fallos en el mismo.

Tema 2. Características de los lenguajes de programación para IA

Cuando un software es muy complejo, introducir nuevos cambios y mejorar las funcionalidades existentes se vuelve cada vez más difícil y costoso.

La **deuda técnica** es una técnica que mide la entropía del software, es decir, el coste acumulado por utilizar código que no cumple con los estándares de calidad y que no mantiene una coherencia con otras partes del programa.

En el largo plazo, una deuda técnica alta puede hacer que los costes de mantenimiento de un software se disparen.

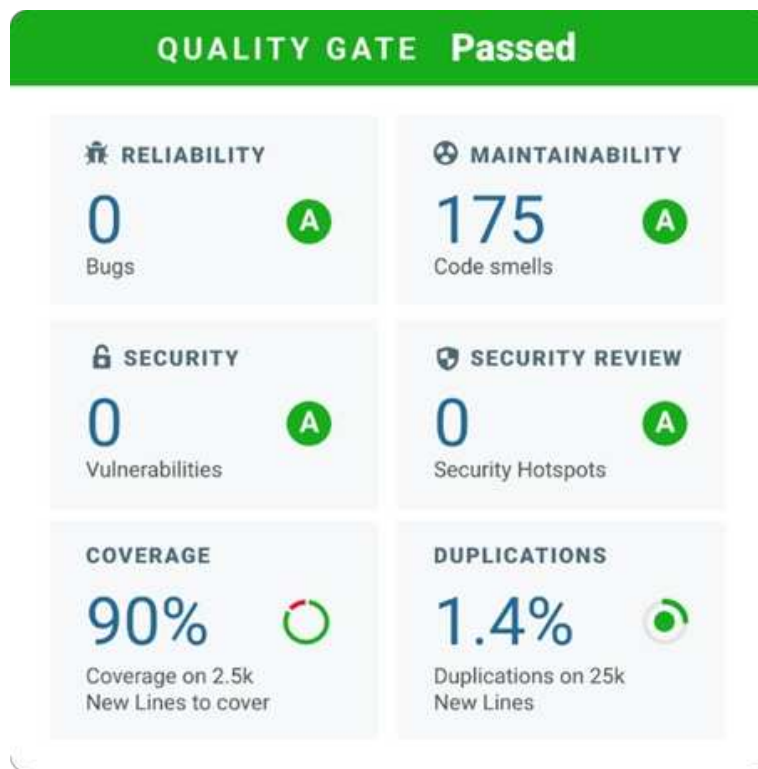


Ilustración 14. Ejemplo de resultados de calidad en la plataforma SonarCloud. Fuente:

<https://sonarcloud.io/>

Tema 2. Características de los lenguajes de programación para IA

► Funciones puras

Las **funciones puras** son también conocidas como funciones libres de efectos secundarios o funciones sin estado.

Estas funciones cumplen con las siguientes condiciones:

- Retorna el mismo resultado para los mismos argumentos de entrada siempre.
- No tiene efectos secundarios: no modifica datos en base de datos, no altera el estado de variables externas, etc.

Un ejemplo de función pura puede ser una función suma, que recibe dos parámetros y devuelve la suma de ambos sin modificar ninguno de ellos en el proceso.

Por el contrario, una función impura es aquella que tiene efectos secundarios, por ejemplo, modificando los argumentos que recibe en la entrada. Las funciones utilizadas en el paradigma imperativo son generalmente impuras ya que modifican el estado del programa o toman valores de consola diferentes en cada lectura.

► Funciones de orden superior

Las **funciones de orden superior** toman una o más funciones como argumentos en la entrada e incluso pueden retornar una función.

Las funciones de orden superior dan lugar a la **parametrización del comportamiento**, ya que las funciones representan un comportamiento y puede ser pasado como parámetro a su vez a otras funciones.

Tema 2. Características de los lenguajes de programación para IA

Ejemplo de programación funcional en Java

```
Optional<Car> carOpt = this.carService.findById(id);  
return carOpt.map(  
    car -> ResponseEntity.ok(car))  
    .orElseGet(  
        () -> ResponseEntity.notFound().build()  
    );
```

Ilustración 15. Ejemplo de controlador REST que devuelve un objeto Coche o una respuesta HTTP Not Found en función de si existe en la base de datos o no.

Programación lógica

El paradigma de la **programación lógica** utiliza un enfoque declarativo basado en la lógica formal.

Se compone de **hechos** y **reglas** en un determinado ámbito de dominio o problema en lugar de utilizar instrucciones. Por ejemplo:

A es verdadero si B, C y D son verdaderos.

Dentro de los lenguajes del paradigma de la programación lógica destaca **Prolog**, abreviatura de *programming in logic*.

Tema 2. Características de los lenguajes de programación para IA

2.5. Características de los lenguajes de programación para IA

Los **lenguajes de programación** son la herramienta principal para explorar y construir programas informáticos en el área de la inteligencia artificial.

Los **programas de inteligencia artificial (IA)** llevan a cabo tareas como el aprendizaje, razonamiento, reconocimiento de información simbólica en contextos, etc.

En las primeras etapas, los lenguajes de programación se utilizaban principalmente para realizar **cálculos numéricos**.

Con el fin de construir programas para simular la inteligencia humana es necesario definir algoritmos capaces de procesar **información de cualquier tipo**. Para ello, es necesario trabajar con símbolos o información que no sean únicamente datos numéricos.

La **computación simbólica** consiste en el desarrollo de algoritmos que permiten manipular expresiones y símbolos matemáticos. Estos símbolos son combinados con reglas o heurísticas para llevar a cabo resolución de problemas y procesar información. Algunos campos en los que se utiliza la computación simbólica son:

- ▶ Álgebra computacional
- ▶ Resolución de teoremas
- ▶ Sistemas de planificación
- ▶ Diagnóstico
- ▶ Reescribir sistemas

Tema 2. Características de los lenguajes de programación para IA

- ▶ Representación del conocimiento
- ▶ Sistemas expertos

La **simulación de inteligencia** se divide en diferentes ramas o subproblemas de forma que sea posible crear sistemas inteligentes. Las más importantes son:

- ▶ Razonamiento y resolución de problemas
- ▶ Representación del conocimiento
- ▶ Planificación
- ▶ Aprendizaje automático (Machine Learning)
- ▶ Procesamiento del lenguaje natural (PLN)
- ▶ Percepción: visión por computador, reconocimiento facial, de voz, de objetos, etc.

Entre las **características** necesarias para que un lenguaje de programación sea idóneo para inteligencia artificial destacan:

- ▶ Paradigma declarativo
- ▶ Manejo de estructuras de datos
- ▶ Búsqueda
- ▶ Optimización
- ▶ Lógica
- ▶ Métodos probabilísticos
- ▶ Manejo de incertidumbre
- ▶ Clasificación

Tema 2. Características de los lenguajes de programación para IA

- ▶ Estadística
- ▶ Recursión y Back tracking
- ▶ Aprovechamiento de los recursos computacionales (CPU, GPU, etc.)

Por lo general, los problemas de inteligencia artificial tienden a ser muy **específicos del dominio** en cuestión sobre el que operan (sector bancario, sector sanitario, marketing, etc.). Por ello, las estrategias heurísticas o de resolución de problemas deben desarrollarse empíricamente a través de enfoques de **generación y prueba** o ensayo y error.

La **programación de IA** difiere notablemente de los enfoques de ingeniería de software estándar donde la programación generalmente comienza a partir de una especificación formal detallada. En la programación de IA, la propia **implementación** es en realidad parte del proceso de **especificación** del problema.

Debido a la **naturaleza difusa** de muchos de los problemas de IA, la programación de IA se beneficia notablemente cuando se utilizan lenguajes de programación que liberan al programador de la mayoría de los aspectos técnicos (asignación y liberación de espacio en memoria, construcción de tipos de datos, etc.).

Según esto, el **paradigma declarativo** beneficia a la programación de IA al permitir crear programas sin necesidad de especificar todos los pasos para ejecutar el flujo de control, simplemente poder especificar qué es lo que se necesita.

Dentro de la programación declarativa se encuentra la **programación funcional** y la **programación lógica**. Ambos paradigmas se basan en formalismos matemáticos, y los lenguajes de programación más conocidos en este ámbito son **Lisp, Prolog y Haskell**.

Tema 2. Características de los lenguajes de programación para IA

En la actualidad muchos lenguajes soportan **múltiples paradigmas de programación**, pudiendo ser utilizados tanto para el desarrollo de software Enterprise como para el desarrollo de software de IA.

Otro factor clave es la existencia de **frameworks** y **librerías** en constante evolución para el desarrollo de software de IA como es el caso de los lenguajes **Python** y **Java**.

El proceso de resolución de problemas a través de la **ciencia de datos** involucra una serie de pasos:

- ▶ Acceso y extracción de los datos
 - Sensores
 - Archivos
 - Bases de datos
 - APIs
- ▶ Análisis de los datos
 - Exploración de los datos
 - Limpieza y tratamiento de datos
- ▶ Preprocesamiento de los datos
- ▶ Modelado
- ▶ Visualización e interpretación de resultados

Tema 2. Características de los lenguajes de programación para IA

- ▶ Despliegue
 - Aplicaciones de escritorio
 - Sistemas Enterprise
 - Sistemas embebidos

Un **lenguaje de programación** que sea útil en las anteriores áreas es una buena elección para la resolución de problemas de inteligencia artificial.

Existen cientos de lenguajes de programación, pero solo algunos contribuyen significativamente en la resolución de problemas de inteligencia artificial.

Con el exponente crecimiento de las tecnologías de **Big Data** e **Inteligencia Artificial** es común utilizar más de un lenguaje de programación en una misma arquitectura software.

Se utilizan entonces diferentes lenguajes de programación y herramientas para cada una de las fases requeridas en los proyectos de Inteligencia Artificial.

Desde la captura de los datos hasta el procesamiento de estos y obtención de conclusiones, se desarrollan **flujos de información** donde cada herramienta se encarga de un proceso diferente.

Tema 3. Principales lenguajes de programación para IA

3.1. Introducción y objetivos

Para estudiar este tema lee las Ideas Clave que se presentan a continuación y revisa los contenidos adicionales para mejorar la comprensión y aumentar los conocimientos sobre las materias tratadas.

Este tema supone un recorrido por los diferentes lenguajes de programación vinculados a la Inteligencia Artificial. Objetivos más concretos son:

- ▶ Ser capaz de identificar la existencia de diferentes **lenguajes de programación**.
- ▶ Conocer los distintos lenguajes de programación y las **características** que los componen.
- ▶ Saber distinguir las **fortalezas** de los diferentes lenguajes de programación.
- ▶ Poder decidir el **uso de un lenguaje de programación** u otro en función del problema a tratar.

Tema 3. Principales lenguajes de programación para IA

3.2. Lenguajes de programación para IA

Existen numerosos

- ▶ Lisp
- ▶ Prolog
- ▶ R
- ▶ Python
- ▶ Haskell
- ▶ Java
- ▶ JavaScript
- ▶ C++
- ▶ Julia

Tema 3. Principales lenguajes de programación para IA

3.3. Python

Python es un lenguaje de propósito general que en los últimos años se ha ido haciendo cada vez más popular en áreas como la ciencia de datos o la inteligencia artificial (IA).

Historia

Python fue creado en 1989 por Guido van Rossum. Guido empezó a implementar este lenguaje como pasatiempo con el objetivo de que fuera fácil de usar y aprender, pero, a la vez, que fuese un lenguaje potente.



Figura 1. Guido van Rossum, creador del lenguaje de programación Python. Fuente: <http://perlge-book.com/tag/guido-van-rossum/>

Tema 3. Principales lenguajes de programación para IA

Van Rossum le dio el nombre de «Python» en homenaje al grupo Monty Python del que era fan. Aunque su desarrollo empezó en 1989, la primera versión no se publicaría hasta principios de 1991. Como hemos dicho, el objetivo principal de este lenguaje era que fuese fácil de entender, es decir, que el código que se escribiese con Python fuese más legible que con otros lenguajes de la época como C++ o Java.

Para lograr este objetivo, el desarrollador Tim Peters creó el Zen de Python, que define un conjunto de reglas que representan la filosofía de Python. Todos los usuarios pueden acceder a este conjunto de reglas a través de un easter egg que se introdujo en Python. Para verlo, solo tenemos que ejecutar la instrucción `import this` en una terminal con Python.

Siguiendo la filosofía propuesta en el Zen de Python, se creó una guía de estilo que se encuentra descrita en el Python Enhancement Proposal, abreviado como PEP, versión 8. Esta guía contiene las reglas de estilo que se aplicaron a la hora de desarrollar Python para que se sigan en el desarrollo de nuevas aplicaciones.

PEP 8 -- Style Guide for Python Code

PEP:	8
Title:	Style Guide for Python Code
Author:	Guido van Rossum <guido at python.org>, Barry Warsaw <barry at python.org>, Nick Coghlan <ncoghlan at gmail.com>
Status:	Active
Type:	Process
Created:	05-Jul-2001
Post-History:	05-Jul-2001, 01-Aug-2013

Figura 2. Información de la guía de estilos de Python PEP8. Fuente: <https://www.python.org/dev/peps/pep-0008/>

Tema 3. Principales lenguajes de programación para IA

Ventajas de Python

Python tiene varias propiedades que lo han convertido en un lenguaje muy potente y fácil de aprender. Estas propiedades son las siguientes:

- ▶ **Tipado dinámico:** Python no necesita que definamos el tipo de las variables cuando las inicializamos como pasa, por ejemplo, en Java o C. Cuando inicializamos una variable Python le asigna el tipo del valor que le estamos asignando. Incluso, durante la ejecución, una misma variable podría contener valores con distintos tipos. Esta propiedad hace que sea más sencillo aprender a usarlo, aunque hace que sea más difícil detectar errores asociados con los tipos de datos.
- ▶ Lenguaje **multiparadigma:** Python permite aplicar diferentes paradigmas de programación como son la programación orientada a objetos, como Java o C++, programación imperativa, como C, o programación funcional, como Haskell.
- ▶ **Interpretado/scripts:** otra ventaja es que podemos ejecutar Python de forma interpretada o usando scripts. Es decir, puedo abrir una consola de Python y escribir y ejecutar las instrucciones una a una o, por otro lado, puedo crear un fichero que almacene todo el programa.
- ▶ **Extensible:** por último, Python cuenta con una gran cantidad de módulos y librerías que podemos instalar para incluir nuevas funcionalidades. Sin embargo, como Python está implementado usando C++, podemos crear nuevos módulos en C++ e incluirlo en Python haciendo que el lenguaje sea extensible a nuevos módulos.

Versiones en Python

Uno de los problemas que tenía una persona que se inicializaba en Python era saber qué versión tenía que instalar, ya que hasta finales de diciembre de 2019 existían dos versiones activas.

Tema 3. Principales lenguajes de programación para IA

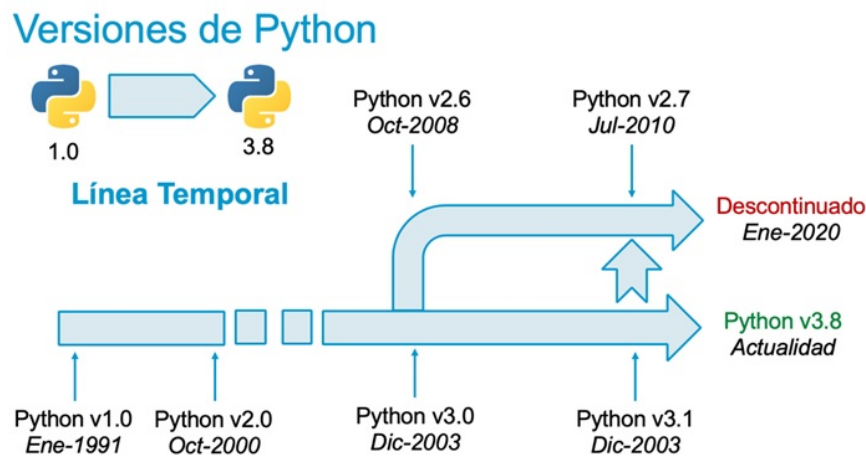


Figura 3. Línea temporal de las versiones de Python.

La primera versión de Python se publicó en 1991, desde entonces se han publicado varias versiones que han seguido siendo más o menos retrocompatibles. Es decir, podía usar esas versiones en programas que había creado en versiones anteriores.

Sin embargo, a finales de 2008 se iba a publicar la versión 3.0. Esta nueva versión era un cambio radical con respecto a las predecesoras y esto hacía que no fuera compatible con las versiones anteriores de Python. Por este motivo, se publicó la versión 2.6 como soporte para los desarrollos de la versión 2. En la versión 2.6 se incluyeron nuevas funcionalidades de la versión 3, pero adaptadas a la versión 2. Desde ese momento Python contaba con 2 versiones y las novedades las tenían que duplicar en ambas. Por ejemplo, en 2010 publicaron la versión 3.1 y la 2.7 que incluía las nuevas funcionalidades, pero adaptadas a la versión 2.

Sin embargo, desde el equipo de Python siempre han explicado que las versiones 2.6 o 2.7 eran un parche y que no iban a ser versiones funcionales en un futuro. Este hecho se hizo oficial en enero de 2020 cuando se decidió que la versión 2.7 quedaba descontinuada y que a partir de entonces solo se publicarían nuevas funcionalidades para la versión 3.

Tema 3. Principales lenguajes de programación para IA

3.4. R

R es uno de los lenguajes de programación más populares para el modelado estadístico y el análisis de datos.



Figura 4: Logo de lenguaje de programación R. Fuente: dominio público.

R fue conceptualizado en los Laboratorios Bell por John Chambers en 1976. Este lenguaje fue desarrollado como una extensión, así como una implementación del lenguaje de programación S.

El proyecto R fue desarrollado por Ross Ihaka y Robert Gentleman y lanzado en 1992, su primera versión en 1995 y una versión beta estable en el año 2000.

R es un conjunto integrado de herramientas de software para manipulación de datos, cálculo y gráficos monitor. Algunos de los aspectos clave de R son:

- ▶ Conjunto de operadores para cálculos en matrices.
- ▶ Colección amplia, coherente e integrada de herramientas intermedias para el análisis de datos.

Tema 3. Principales lenguajes de programación para IA

- ▶ Facilidades gráficas para el análisis y visualización de datos
- ▶ Lenguaje de programación bien desarrollado, simple y eficaz (llamado "S") que incluye condicionales, bucles, funciones recursivas definidas por el usuario y facilidades de entrada y salida. (La mayoría de las funciones proporcionadas por el sistema están escritas en el lenguaje S.)

El término “entorno” o “environment” hace que R sea un sistema totalmente integrado, en lugar de una acumulación de herramientas muy específicas e inflexibles, como suele ser el caso con otro software de análisis de datos.

R es en gran medida un vehículo para nuevos métodos de análisis de datos interactivos en desarrollo.

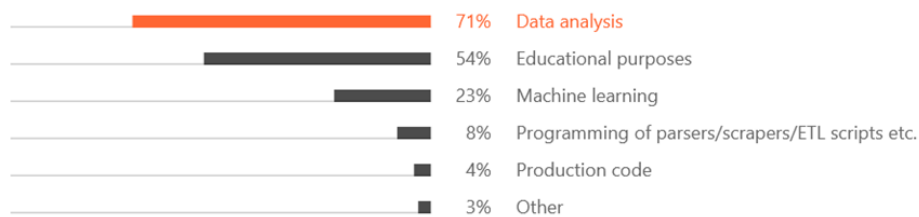


Figura 5: Usos del lenguaje de programación R en la actualidad. Fuente: [JetBrains Devecosystem 2021](#).

Tema 3. Principales lenguajes de programación para IA

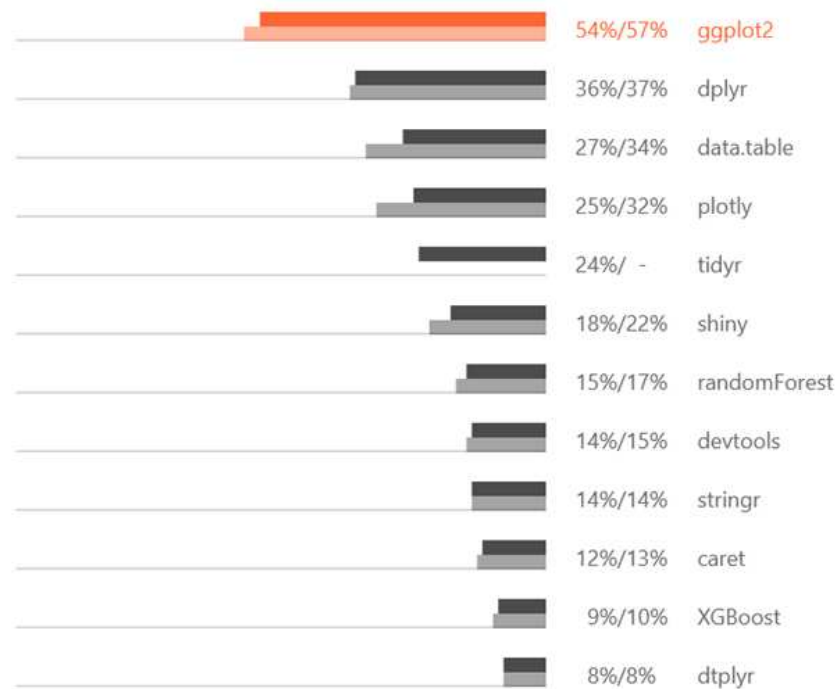


Figura 6: Paquetes más populares para el lenguaje de programación R. Fuente: [JetBrains Devecosystem](https://www.jetbrains.com/devecosystem/) 2021.

Ventajas

R tiene una serie de **ventajas** que lo hacen idóneo para el entorno de tratamiento y análisis de datos:

- ▶ Open Source
- ▶ Buen soporte para la manipulación de datos
- ▶ Amplio ecosistema de paquetes: <https://cran.r-project.org/>
- ▶ Capacidad para visualización de datos
- ▶ Compatibilidad
- ▶ Generación de reportes

Tema 3. Principales lenguajes de programación para IA

- ▶ Portabilidad
- ▶ Útil para proyectos de estadística
- ▶ Ideal para proyectos de aprendizaje automático o Machine Learning
- ▶ Recibe actualizaciones de manera constante

Tema 3. Principales lenguajes de programación para IA

3.5. Java

La tecnología **Java** consiste en un lenguaje de programación y también en una plataforma para el desarrollo de software portable entre sistemas operativos.



Figura 7: Logo de lenguaje de programación Java. Fuente: dominio público.

Surge a comienzos de los años 90 como una iniciativa de la empresa Sun Microsystems para cubrir la necesidad de creación de software para dispositivos inteligentes. En 2010 Oracle toma el relevo de su desarrollo adquiriendo la empresa Sun.

Tema 3. Principales lenguajes de programación para IA



Figura 8: Logo la empresa Sun Microsystems, empresa original en la que surgió el lenguaje de programación Java. Fuente: dominio público.



Figura 9: Logo de Oracle, empresa que desarrolla el lenguaje de programación Java en la actualidad. Fuente: dominio público.

Java es un lenguaje de programación de alto nivel y multiparadigma, es decir, soporta múltiples paradigmas de programación: estructurada, orientada a objetos, funcional.

Es rápido, seguro y fiable, permitiendo a los desarrolladores crear diferentes tipos de aplicaciones software, entre los que destacan:

- ▶ **Aplicaciones standalone:** software que puede ser ejecutado sin ser instalado y que puede trabajar offline.
- ▶ **Aplicaciones web:** software cliente-servidor accedido comúnmente desde un navegador.
- ▶ **Aplicaciones móviles y de internet de las cosas (IoT):** aplicaciones para dispositivos inteligentes.

Tema 3. Principales lenguajes de programación para IA

Principales características del lenguaje Java:

- ▶ **Fácil de aprender**: con respecto a otros lenguajes como C y C++.
- ▶ **Robusto**: tipado fuerte y mecanismos para gestión de excepciones.
- ▶ **Arquitectura neutral**: gracias a la Máquina Virtual Java se puede ejecutar sobre cualquier Plataforma.
- ▶ **Capacidad multihilo**: el propio lenguaje ofrece la capacidad para crear múltiples hilos para ejecutar diferentes tareas simultáneamente.

El código se escribe en ficheros con **extensión .java**, que después son compilados a código intermedio en ficheros con **extensión .class** por el compilador javac.

El código intermedio se conoce como **bytecode** y es interpretado y compilado a código nativo del sistema operativo por la Máquina Virtual de Java.

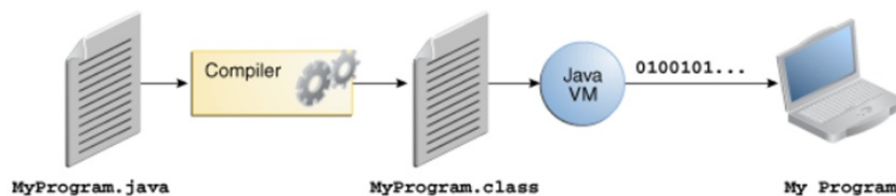


Figura 10: Proceso de compilación y ejecución código Java. Fuente: [Oracle](#).

Tema 3. Principales lenguajes de programación para IA

La plataforma Java es un conjunto de librerías y herramientas para el desarrollo de software.

Se compone de los siguientes elementos:

Bibliotecas estándar o Java Application Programming Interface (API): un conjunto de interfaces y clases con funcionalidades comunes ya desarrolladas para ser utilizadas.

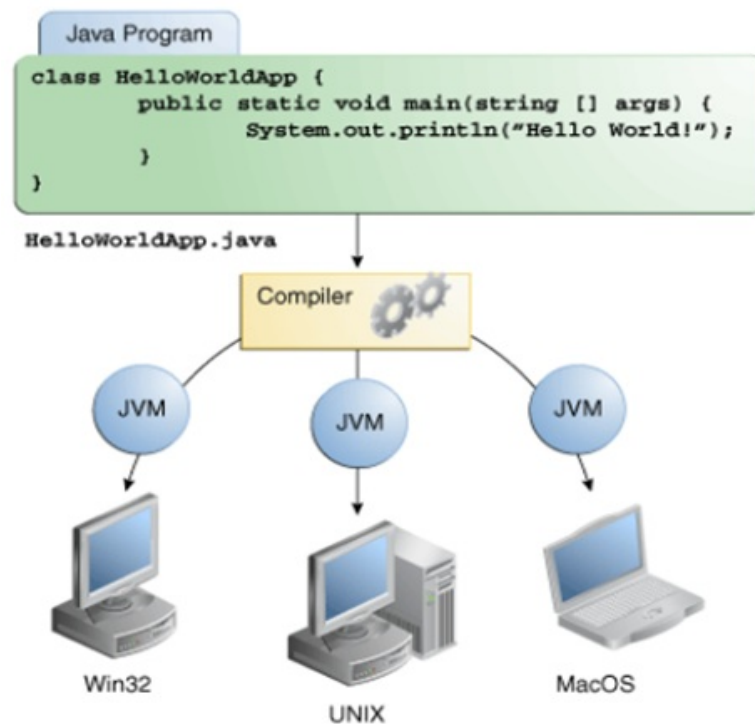
Máquina Virtual Java (JVM): compuesta por los registros, la pila, el recolector de basura, etc. Permite interpretar el *bytecode* creado por el compilador de Java.

La máquina Virtual Java

La **Máquina Virtual Java** es el componente principal de la plataforma Java.

Una de las filosofías principales de Java es la portabilidad entre sistemas. Gracias a la Máquina Virtual Java es posible escribir un código una vez y ser ejecutado en cualquier sistema operativo, transformándose al código nativo correspondiente.

Tema 3. Principales lenguajes de programación para IA



Ediciones de la plataforma Java

Existen múltiples ediciones de la plataforma Java según el tipo dispositivo final sobre el que se ejecutará el software desarrollado, siendo las más populares en la actualidad:

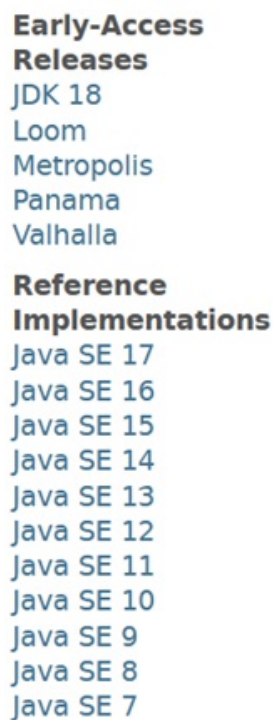
- ▶ **Java SE o Java Standard Edition** : dirigido a estaciones de trabajo y ordenadores de propósito general.
- ▶ **Java EE o Java Enterprise Edition** : ahora conocida como *Jakarta Enterprise Edition*, es un conjunto de librerías añadidas sobre Java SE para incorporar funcionalidades dirigidas a servidores empresariales y entornos distribuidos en Internet, acceso a bases de datos, creación de servicios web, etc.

Tema 3. Principales lenguajes de programación para IA

Versiones Java

Desde el lanzamiento de su versión 9, Java ha adoptado un ciclo de release en el que cada **6 meses** se libera una nueva versión.

Aquellas versiones establecidas como **LTS** o *Long Term Support* son las más recomendadas por ofrecer mantenimiento y actualizaciones a largo plazo.



El diagrama muestra la jerarquía de las versiones de Java. Se divide en dos categorías principales: 'Early-Access Releases' y 'Reference Implementations'. La primera categoría incluye JDK 18, Loom, Metropolis, Panama y Valhalla. La segunda categoría incluye Java SE 17, Java SE 16, Java SE 15, Java SE 14, Java SE 13, Java SE 12, Java SE 11, Java SE 10, Java SE 9, Java SE 8 y Java SE 7.

Early-Access Releases
JDK 18
Loom
Metropolis
Panama
Valhalla

Reference Implementations
Java SE 17
Java SE 16
Java SE 15
Java SE 14
Java SE 13
Java SE 12
Java SE 11
Java SE 10
Java SE 9
Java SE 8
Java SE 7

Figura 12: versiones de Java disponibles. Fuente: [Open JDK](#).

Tema 3. Principales lenguajes de programación para IA

Popularidad de Java

Desde su **aparición en 1995** Java se ha convertido en una de las tecnologías más populares para el desarrollo de software, manteniéndose entre los primeros puestos año tras año.

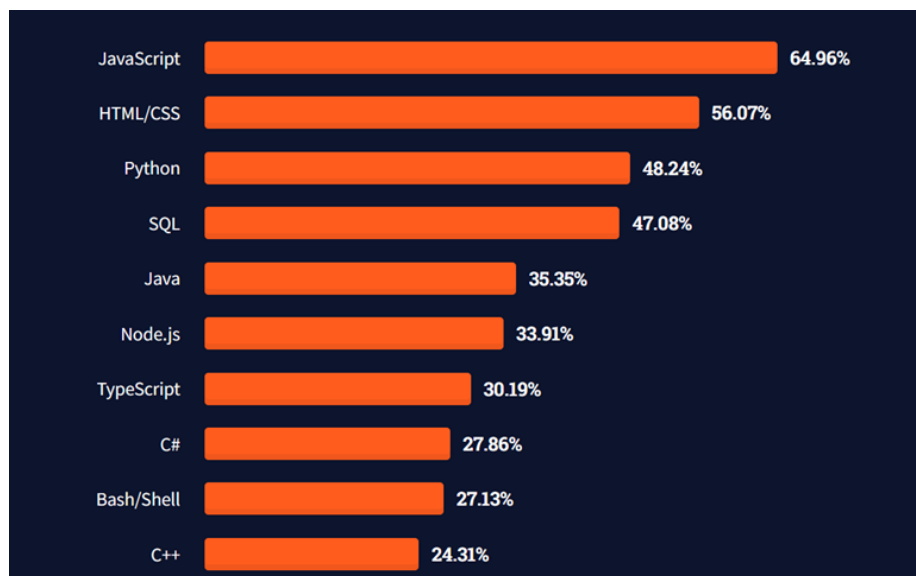


Figura 13: 10 tecnologías más populares según la encuesta [Stack Overflow Developer Survey 2021](#).

Nov 2021	Nov 2020	Change	Programming Language	Ratings	Change
1	2	▲	 Python	11.77%	-0.35%
2	1	▼	 C	10.72%	-5.49%
3	3		 Java	10.72%	-0.96%

Figura 14: los 3 lenguajes de programación más populares. Fuente: Índice [TIOBE](#).

Tema 3. Principales lenguajes de programación para IA

En la actualidad Java se emplea mayoritariamente para el desarrollo de software empresarial, sitios web y herramientas de utilidad. Entre los numerosos usos de Java se encuentra la ciencia de datos y el aprendizaje automático:

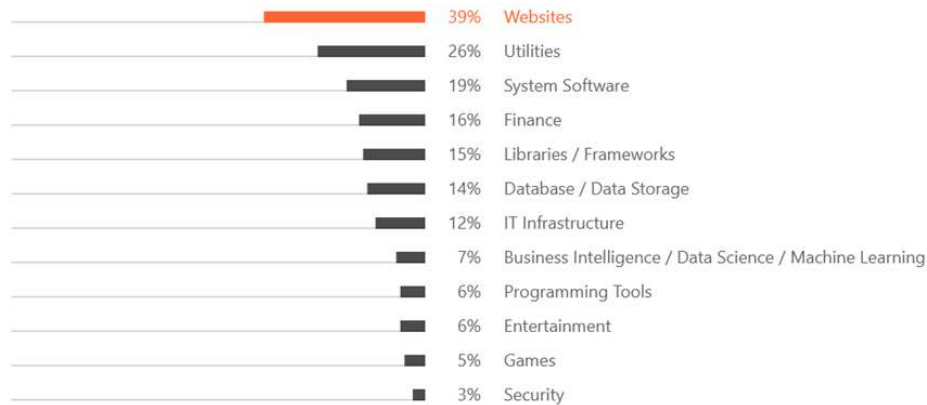


Figura 15: Principales usos de Java en la actualidad. Fuente: [JetBrains Devecosystem 2021](#).

Tema 3. Principales lenguajes de programación para IA

3.6. Lisp



Figura 16: Logo de lenguaje de programación Lisp. Fuente: dominio público.

Lisp es un lenguaje de programación bajo el paradigma de la programación funcional, de propósito general, se basa en la aplicación de funciones a los datos y se apoya en la utilización de funciones matemáticas para el control de estos.

El elemento fundamental en Lisp es la lista, una estructura de datos. Cada función de Lisp y cada programa que genera viene dado en forma de lista.

Lisp es un lenguaje de programación creado en los años 50 por John McCarthy con el objetivo de desarrollar un lenguaje que mostrara la información estructurada en listas en las que permitieran gestionarla. De ahí el nombre que le dio a este lenguaje, LISP (List-Processing), porque fue creado principalmente para el procesamiento de listas.

Con el transcurso del tiempo Lisp dio origen a varios dialectos. Entre los más importantes podemos citar a: Scheme, Common Lisp, Clojure, Emacs Lisp, Racket. De todos ellos el Common Lisp se está consolidando como el estándar.

Tema 3. Principales lenguajes de programación para IA

Lisp es considerado en la actualidad como un modelo de referencia para la programación funcional.

Características

Lisp al ser un lenguaje funcional, se basa en la utilización de funciones matemáticas para el control de los datos. Lisp diferencia dos tipos de elementos básicos:

- ▶ El **átomo**, datos elementales de varios tipos como números, símbolos, caracteres y cadenas de caracteres.
- ▶ Las **listas**: dentro destaca la lista “nil”, que es una lista nula que no tiene ningún elemento.

Lisp trata a los elementos o parámetros en las funciones de manera no destructiva, de forma que la mayoría de las funciones devuelven una lista que es el resultado de alguna transformación de otra que recibió, pero sin modificar los parámetros recibidos.

Cabe destacar que Lisp fue uno de los primeros lenguajes en manejar las excepciones con los comandos catch y throw. La **gestión de excepciones** es un pilar fundamental en la creación de programas robustos que puedan gestionar errores de forma correcta sin producir fallos que impidan su utilización.

Los datos en Lisp se organizan:

- ▶ Átomos literales (símbolos)
- ▶ Átomos numéricos
- ▶ La estructura de datos básica es la lista. Incluye primitivas para su manipulación.

Tema 3. Principales lenguajes de programación para IA

- ▶ Los comentarios comienzan por ‘;’
- ▶ Los parámetros de función van todos por valor o por referencia según la clasificación de la función.

Una de las razones por las que Lisp está especialmente vinculado a la programación en inteligencia artificial (IA), es precisamente porque su código y todos los datos tienen la misma estructura, en forma de lista.

Lisp se utilizó como lenguaje de programación en el sistema de inteligencia artificial SHRDLU, un programa informático de comprensión del lenguaje natural (CLN).

Tema 3. Principales lenguajes de programación para IA

3.7. JavaScript

JavaScript (abreviado JS) es un lenguaje de programación de alto nivel y multiparadigma creado para dotar de dinamismo a las páginas web, evitando así una apariencia estática.

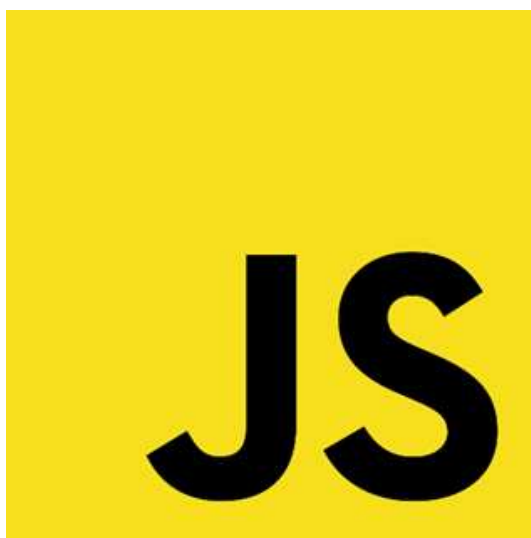


Figura 17: Logo de lenguaje de programación JavaScript. Fuente: dominio público.

Los programas desarrollados en JavaScript se llaman scripts y se pueden escribir directamente sobre documentos HTML o en archivos externos con extensión .js, siendo ejecutados por el navegador cuando se carga la página web.

Actualmente JavaScript es soportado por todos los navegadores web, lo que lo convierte en el lenguaje de programación idóneo para ser ejecutado junto a HTML y CSS para crear páginas web dinámicas, permitiendo mejoras en la interfaz de usuario.

JavaScript es un lenguaje de programación multiparadigma en cuya sintaxis podemos encontrar: variables, operadores, funciones, estructuras de datos, estructuras de control condicional, estructuras de control repetitivo...

Tema 3. Principales lenguajes de programación para IA

En los orígenes se llamaba LiveScript, pero se renombró a JavaScript debido a que por aquella época el lenguaje Java era muy popular, pero conviene destacar que Java y JavaScript son lenguajes completamente diferentes.

```
/**
 * Get CSS computed property of the given element
 * @method
 * @memberof Popper.Utils
 * @argument {Element} element
 * @argument {String} property
 */
function getStyleComputedProperty(element, property) {
  if (element.nodeType !== 1) {
    return [];
  }
  // NOTE: 1 DOM access here
  var window = element.ownerDocument.defaultView;
  var css = window.getComputedStyle(element, null);
  return property ? css[property] : css;
}

/**
 * Returns the parentNode or the host of the element
 * @method
 * @memberof Popper.Utils
 * @argument {Element} element
 * @returns {Element} parent
 */
function getParentNode(element) {
  if (element.nodeName === 'HTML') {
    return element;
  }
  return element.parentNode || element.host;
}
```

Figura 18: Ejemplo de código JavaScript.

La idea principal detrás de la creación de este lenguaje fue sobrepasar la limitación que había en aquella época de que las webs fueran estáticas una vez se cargaban en el navegador.

Tema 3. Principales lenguajes de programación para IA

Versiones

JavaScript fue desarrollado en 1995 por Netscape, empresa de software famosa por el navegador popular Netscape navigator.

En 1996 ECMA lo empieza a evaluar como una especificación para todos los navegadores y en 1997 se declara estándar (ECMA-262).

A partir de 2016 se empiezan a lanzar versiones cada año, utilizando el número de año como nombre de versión, en lugar de continuar la numeración ES7, ES8, ES9, ES10, etc.

ECMAScript 1 (ES1)	1997
ECMAScript 2 (ES2)	1998
ECMAScript 3 (ES3)	1999
ECMAScript 4 (ES4)	No se publicó.
ECMAScript 5 (ES5)	2009
ECMAScript 6 (ES6)	2015
ECMAScript 2016	2016
ECMAScript 2017	2017
ECMAScript 2018	2018
ECMAScript 2019	2019
ECMAScript 2020	2020

Figura 19: Tabla de versiones JavaScript.

Tema 3. Principales lenguajes de programación para IA

Machine Learning con JavaScript

TensorFlow

TensorFlow es una biblioteca de código abierto que sirve para desarrollar y entrenar modelos de aprendizaje automático. TensorFlow es una plataforma de extremo a extremo para el aprendizaje automático o machine learning.



Figura 20: Logo de la librería TensorFlow.cFuente: <https://www.tensorflow.org/>

TensorFlow es una biblioteca de código abierto que sirve para desarrollar y entrenar modelos de aprendizaje automático. TensorFlow es una plataforma de extremo a extremo para el aprendizaje automático o machine learning.

Gracias a su ecosistema integral y flexible de herramientas, bibliotecas y recursos de la comunidad, TensorFlow ayuda a los investigadores a innovar con el aprendizaje automático y a los desarrolladores a crear e implementar aplicaciones con tecnología de aprendizaje automático de forma fácil.

Dentro del proyecto TensorFlow existe TensorFlow.js, una biblioteca de JavaScript para entrenar e implementar modelos en el navegador y en [Node.js](https://nodejs.org/).

TensorFlow.js permite utilizar modelos listos para usar de JavaScript, o convertir modelos de TensorFlow para Python.

Privacidad y personalización

Otra razón por la que JavaScript es una opción interesante para Machine Learning es la personalización de modelos. Por ejemplo, desarrollar un modelo de aprendizaje

Tema 3. Principales lenguajes de programación para IA

automático de generación de texto que se adapte a las preferencias de idioma de cada usuario. Una solución sería almacenar un modelo por usuario en el servidor y entrenarlo con los datos del usuario. Esto supondría una carga adicional en los servidores a medida que el número de usuarios de la plataforma crece y también requeriría almacenar datos potencialmente confidenciales en la nube.

Una alternativa sería crear un modelo base o plantilla en el servidor, crear una copia en el dispositivo del usuario y ajustar el modelo con los datos del usuario mediante bibliotecas de aprendizaje automático de JavaScript.

Por un lado, esto mantendría los datos en los dispositivos de los usuarios y evitaría la necesidad de enviarlos al servidor. Por otro lado, liberaría los recursos del servidor al evitar enviar cargas adicionales de inferencia y entrenamiento a la nube. De esta forma, los usuarios podrían utilizar las capacidades de aprendizaje automático incluso cuando estén desconectados de los servidores, es decir, en entornos offline.

Dispositivos móviles

Otro beneficio del aprendizaje automático de JavaScript es la fácil integración con aplicaciones móviles. El soporte de Python en sistemas operativos móviles aún se encuentra en las etapas preliminares. Pero ya existe un amplio conjunto de herramientas de desarrollo de aplicaciones móviles JavaScript multiplataforma, como, por ejemplo:

- ▶ [Apache Cordova](#)
- ▶ [Ionic](#)
- ▶ [Flutter](#)
- ▶ [React Native](#)

Estas herramientas se han vuelto muy populares porque permiten escribir un mismo código base una vez e implementarlo para dispositivos iOS y Android. Para que el

Tema 3. Principales lenguajes de programación para IA

código sea compatible en diferentes sistemas operativos, las herramientas de desarrollo multiplataforma lanzan una "vista web", un objeto de navegador que puede ejecutar código JavaScript y puede integrarse en una aplicación nativa del sistema operativo de destino. Estos objetos de navegador son compatibles con las bibliotecas de aprendizaje automático de JavaScript.

Una excepción es React Native, un popular marco de desarrollo de aplicaciones móviles multiplataforma que no depende de la vista web para ejecutar aplicaciones. Sin embargo, dada la popularidad de las aplicaciones móviles de aprendizaje automático, Google ha lanzado una versión especial de TensorFlow.js para React Native.

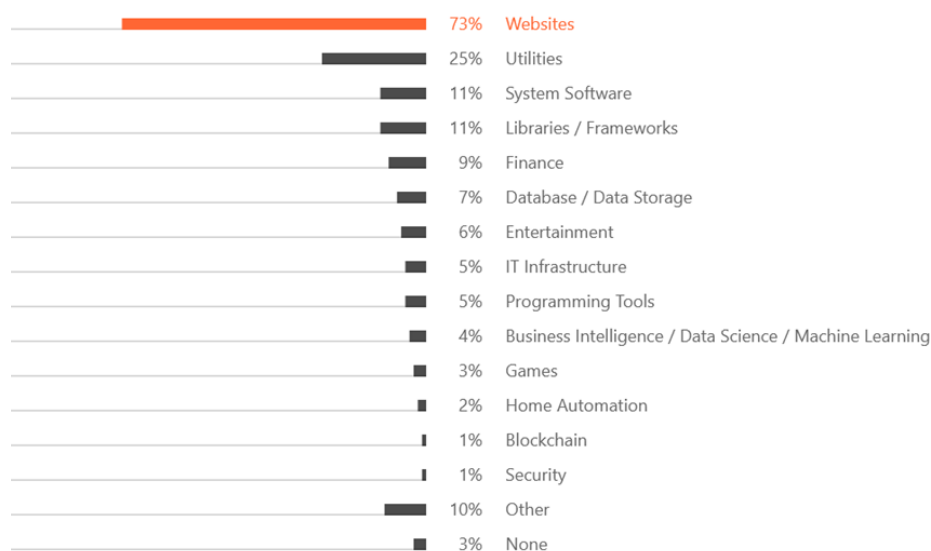


Figura 21: Principales usos de JavaScript en la actualidad. Fuente: [JetBrains Devecosystem 2021](#).

Tema 3. Principales lenguajes de programación para IA

3.8. Prolog



Figura 22: Logo de lenguaje de programación Prolog. Fuente: dominio público.

Prolog es un lenguaje de programación lógico y semi-interpretado. La programación lógica es un paradigma de los lenguajes de programación en el cual los programas se consideran como una serie de aserciones lógicas. La programación lógica es un subparadigma del paradigma declarativo.

Prolog es un lenguaje de programación basado en reglas, desarrollado en la Universidad de Aix-Marseille (Marsella, Francia) por los profesores Alain Colmerauer y Philippe Roussel, como una herramienta práctica para programación lógica.

Nació de un proyecto que no tenía como objetivo la implementación de un lenguaje de programación, sino el procesamiento de lenguajes naturales.

El funcionamiento de Prolog es muy similar a Java. El código fuente se compila a un código objeto o *bytecode* el cuál se interpreta en una máquina virtual denominada Warren Abstract Machine (comúnmente denominada WAM).

Por eso, un entorno de desarrollo Prolog se compone de:

- Un **compilador**: Transforma el código fuente en código de byte. A diferencia de Java, no existe un Standard al respecto. Por eso, el código de byte generado por un entorno de desarrollo no tiene por qué funcionar en el intérprete de otro entorno.

Tema 3. Principales lenguajes de programación para IA

- ▶ Un **intérprete**: Ejecuta el código de byte. Se trata de una utilidad que permite probar los programas, depurarlos, etc. Su funcionamiento es similar a los interfaces de línea de comando de los sistemas operativos.
- ▶ Una **biblioteca de utilidades**: similar a la biblioteca estándar de Java. Muchos entornos incluyen unas bibliotecas standard-ISO que permiten funcionalidades básicas como manipular cadenas, entrada/salida, etc.

Generalmente, los entornos de desarrollo ofrecen extensiones al lenguaje como pueden ser la programación con restricciones, concurrente, orientada a objetos, etc.

SICStus, CIAO Prolog, y posiblemente otros más, ofrecen entornos integrados generalmente basados en Emacs que resultan muy fáciles de usar. CIAO Prolog además ofrece un auto documentador similar al existente para Java además de un preprocesador de programas. Prácticamente todos ellos son multiplataforma.

Elementos en Prolog

Para construir programas informáticos en Prolog es necesaria una serie de elementos:

- ▶ **Átomos**: es una definición genérica de un objeto del mundo que se quiere representar.
- ▶ **Predicados**: permite especificar características de los objetos del mundo real o las relaciones entre ellos.

Hechos

Es algo que está ocurriendo en el mundo, característica o relación entre objetos. En el lenguaje natural un hecho podría ser por ejemplo que Algoritmos y Estructuras de datos es una asignatura de Ingeniería Informática. Expresan relaciones entre objetos. Suponiendo que se quiera expresar el hecho de que "un coche tiene ruedas". Este hecho, consta de dos objetos, "coche" y "ruedas", y de una relación llamada "tiene".

Tema 3. Principales lenguajes de programación para IA

La forma de representarlo en PROLOG es: tiene(coche,ruedas).

- ▶ Los nombres de objetos y relaciones deben comenzar con una letra minúscula.
- ▶ Primero se escribe la relación, y luego los objetos separados por comas y encerrados entre paréntesis.
- ▶ Al final de un hecho debe ir un punto (".").

El orden de los objetos dentro de la relación es arbitrario, pero es recomendable seguir una misma nomenclatura y mantener la coherencia a lo largo de la base de hechos.

Variables

No es variable con el concepto que se tiene de ella en la programación habitual. En Prolog, una variable representa el valor de un **Átomo**. Representan objetos que el mismo PROLOG determina.

Una variable puede estar instanciada o no instanciada. Estar instanciada cuando existe un objeto determinado representado por la variable. Los nombres de variables comienzan siempre por una letra mayúscula.

Un caso particular es la variable anónima, representada por el carácter subrayado ("_"). Es una especie de comodín que se utiliza en aquellos lugares que debería aparecer una variable, pero no interesa darle un nombre concreto ya que no va a ser utilizada posteriormente.

Tema 3. Principales lenguajes de programación para IA

Reglas

Las reglas se utilizan en PROLOG para significar que un hecho depende de uno o más hechos. Son la representación de las implicaciones lógicas del tipo $p \rightarrow q$ (p implica q).

- ▶ Una regla consiste en una cabeza y un cuerpo, unidos por el signo ":-".
- ▶ La cabeza está formada por un único hecho.
- ▶ El cuerpo puede ser uno o más hechos (conjunción de hechos), separados por una coma (","), que actúa como el "y" lógico.
- ▶ Las reglas finalizan con un punto (".").

Tema 3. Principales lenguajes de programación para IA

3.9. Otros lenguajes y tecnologías

Los lenguajes anteriormente mencionados son solo algunos de los lenguajes de programación más utilizados para el desarrollo de programas de inteligencia artificial.

También existen otros lenguajes de programación y plataformas relevantes en esta área:

- ▶ C
- ▶ C++
- ▶ MATLAB
- ▶ Julia

Tema 4. Lenguajes de marcado. Información de sus etiquetas

4.1. Introducción y objetivos

Para estudiar este tema lee las Ideas Clave que se presentan a continuación y revisa los contenidos adicionales para mejorar la comprensión y aumentar los conocimientos sobre las materias tratadas.

Este tema supone un recorrido por los diferentes lenguajes de marcado más populares en la actualidad. Objetivos más concretos son:

- ▶ Comprender en qué consisten los **lenguajes de marcado** y sus diferencias con respecto a los lenguajes de programación.
- ▶ Conocer los diferentes **tipos de lenguaje de marcado**.
- ▶ Saber en qué situaciones se puede **aplicar un lenguaje de marcado** u otro.
- ▶ Conocer las **tecnologías relacionadas** con los lenguajes de marcado.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

4.2. Los lenguajes de marcado

Un **lenguaje de marcado** es un lenguaje legible por humanos utilizado para anotar un documento electrónico para que las máquinas puedan comprender mejor el estilo y la estructura del documento.

Un lenguaje de marcado electrónico utiliza **etiquetas** para definir diferentes elementos en un documento. Todos los humanos deben ser capaces de leer lenguajes de marcado, por lo que se basan principalmente en palabras reales inteligibles en lugar de la sintaxis ininteligible utilizada en los lenguajes máquina.

Muchos lenguajes de marcado utilizan conjuntos específicos de instrucciones que explican qué son las diferentes partes de un documento y cómo deben formatearse en la página. En el lenguaje de marcado más común, HTML, esto se hace usando herramientas conocidas como **etiquetas** y **elementos**.

Todos los lenguajes de marcado tienen una cosa en común: utilizan **etiquetas** (es decir, <>) para entrelazar el contenido de un documento con las instrucciones de marcado. Los corchetes angulares son los que marcan las etiquetas y lo que se encuentra entre las etiquetas está representado por el contenido formateado. El marcado puede ser semántico o de presentación, según su posición o uso en un documento.

Una **etiqueta** es responsable de mostrar dónde comienza y termina un elemento. Se utilizan en pares y constan de símbolos menor que (<) y mayor que (>) y divididos por caracteres que especifican las instrucciones. Estos incluyen una barra inclinada hacia atrás cuando las etiquetas se cierran. Una etiqueta de encabezado en HTML, que se usa para mostrar un título importante, sería así:

<title> </title>

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Las etiquetas son responsables de mostrar dónde comienza y termina un elemento.

Un **elemento** incluye etiquetas y lo que está escrito entre ellas. El elemento del encabezado anterior podría ser:

```
<title>Ejemplo de título</title>
```

Algunos de los **lenguajes de marcado más utilizados** son: XML, HTML, XHTML, MathML, SGML, KML. Muchos tienen en común las siglas ML en su nombre, que significa *Markup Language*.

Los lenguajes de marcado funcionan en un segundo plano. Cuando se abre un documento en línea no se ven las etiquetas ni elementos. En su lugar, se ve directamente el título y los textos de los párrafos con el formato que deberían tener, es decir, se visualiza un contenido renderizado.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

4.3. Tipos de lenguajes de marcado

Marcado semántico

Al marcado semántico también se le conoce como **marcado descriptivo**. El marcado semántico explica el **propósito** de un texto en una página, si se trata de un título, un párrafo, etc.

La **semántica** se refiere a la interpretación correcta del significado de una palabra u oración.

Usar una palabra semánticamente es usarla de una manera que esté alineada correctamente con el significado de la palabra. Cuando se usa mal una palabra, no se está usando semánticamente.

Muchas etiquetas HTML tienen un significado semántico. Es decir, el nombre de la etiqueta en sí transmite cierta información sobre el tipo de contenido entre las etiquetas de apertura y cierre.

Por ejemplo, cuando un navegador encuentra un encabezado h1, interpreta que esa etiqueta significa que el contenido del elemento h1 constituye el encabezado más importante de la sección que contiene el elemento. El significado semántico de una etiqueta h1 es que se utiliza para identificar el encabezado más importante de una página web o sección específica.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Marcado de presentación

El marcado de presentación se encarga del aspecto del contenido, indica el formato que tendrán los textos.

El marcado de presentación se encarga de explicar las características visuales específicas del texto. Por ejemplo, `<i>` es la etiqueta que se usa para indicar que un texto debe escribirse en cursiva.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

4.4. HTML



Ilustración 1: lenguaje de marcado HTML. Fuente: dominio público.

HTML son las siglas de **HyperText Markup Language** y se traduce como lenguaje de marcado de hipertexto.

Es el componente más básico en los entornos web. Este lenguaje permite definir el significado y la estructura del contenido web.

‘Hipertexto’ hace referencia a enlaces que conectan páginas entre sí, es decir, una web que contiene enlaces a otras páginas de esa misma web o webs diferentes.

HTML no es un lenguaje de programación, es un **lenguaje de marcado**, la diferencia reside en que simplemente define la estructura de los contenidos, no permite realizar lógica de control ni condicional.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

HTML consiste en una serie de etiquetas especiales que el navegador web interpreta y que permiten indicar qué es cada contenido: párrafos, una lista de elementos, una tabla, una imagen, etc.

Las etiquetas HTML se distinguen por:

- ▶ abrir con el símbolo '<'
- ▶ cerrar con el símbolo '>'

Por un lado, están las etiquetas de apertura, por ejemplo, un párrafo: <p>

Por otro lado, están las etiquetas de cierre, por ejemplo: </p>

Todo lo que esté contenido entre ambas adoptará la estructura que indique el tipo de etiqueta.

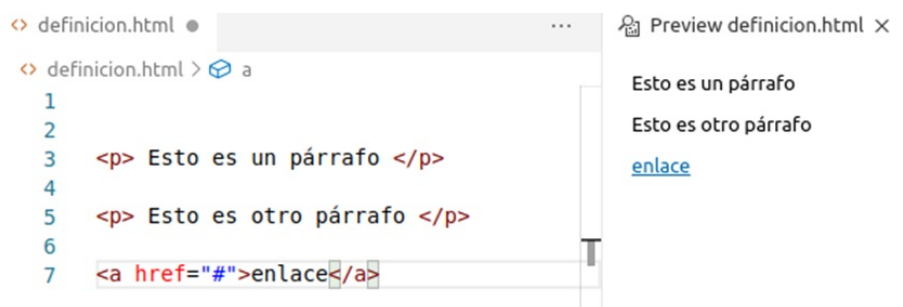


Ilustración 2: etiquetas HTML.

Gracias al lenguaje de marcado HTML se puede definir la estructura de un documento web de manera que los navegadores puedan interpretarla y mostrarla visualmente en forma de una web.

El principal objetivo es que los usuarios de internet puedan navegar fácilmente de una manera visual a través de los contenidos.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

En 1989 Tim Berners Lee inventó la WWW en el CERN (Ginebra, Suiza). La www o World Wide Web es un sistema de distribución de documentos de hipertexto interconectados y accesibles desde internet.

Para definir la estructura de estos documentos Tim Berners Lee crea el lenguaje HTML cuya primera versión oficial HTML 1.0 aparece en 1993.

Desde entonces HTML ha ido evolucionando en diferentes versiones:

HTML 1.0	1993
HTML 2.0	1995
HTML 3.2	1997
HTML 4.0	1998
XHTML	2000
HTML 5.0	2014
HTML 5.1	2016
HTML 5.2	2017

Ilustración 3: versiones de HTML.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Estándar web

HTML es un estándar de la WWW permite que cualquier página web esté escrita en una determinada versión y los navegadores web sean capaces de interpretarlas. El estándar es formulado por:

- ▶ Consorcio WWW o W3C desde 1994 dirigido por Tim Berners-Lee.



Ilustración 4: Logo consorcio WWW. Fuente: dominio público.

- ▶ Web Hypertext Application Technology Working Group o WHATWG desde 2004.



Ilustración 5: logo WHATWG. Fuente: dominio público.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Funcionamiento de la web

Cuando se utiliza un navegador web para acceder a un sitio web a través de una URL ocurre lo siguiente:

- ▶ Se traduce la URL a una dirección IP utilizando la base de datos distribuida de internet conocida como DNS.
- ▶ Se envía una petición HTTP a la dirección IP solicitando el recurso web.
- ▶ El servidor procesa la petición y devuelve un resultado, un documento HTML.
- ▶ El navegador renderiza ese documento HTML de forma visual interpretando el código HTML, CSS y JavaScript para producir el resultado final, una web.

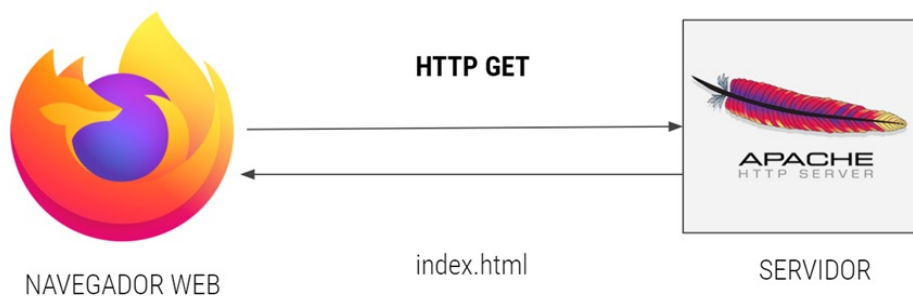


Ilustración 6: Ejemplo de petición HTTP GET.

HTML 5

HTML 5 incorpora nuevas funcionalidades y cambios con respecto a sus predecesores, algunas de las más importantes son:

- ▶ Simplificación en las declaraciones como:
 - Declaración Doctype html.
 - Declaración de codificación de caracteres.
 - Declaración de links a hojas css.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

- Declaración de scripts de javascript.
- ▶ Etiquetas y atributos eliminados.
- ▶ Nuevas etiquetas semánticas: <main>, <header>, <footer>, <nav>, <section>, <aside>, <article>.

Tecnologías web

HTML junto a las tecnologías CSS y JavaScript constituyen los pilares básicos en la creación de páginas web.

CSS o **Cascading Style Sheets** hace referencia a hojas de estilo en cascada y es un lenguaje de diseño gráfico que permite definir la presentación de contenido estructurado mediante un lenguaje de marcado, como por ejemplo HTML.

JavaScript es un lenguaje de programación multiparadigma que se emplea principalmente junto al código HTML y CSS para dotar a una web de comportamiento, es decir, para hacerla más dinámica al poder ejecutar lógica condicional, realizar operaciones, cargar datos desde el servidor, crear código HTML programáticamente, mostrar u ocultar elementos en base al comportamiento del usuario, etc.



Ilustración 7: tecnologías web HTML, JS y CSS. Fuente: dominio público.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Estructura HTML

Para crear un documento HTML utilizando la última versión HTML 5 se crea un archivo con extensión .html y con un entorno de desarrollo (IDE) se escribe el código HTML.

Existen multitud de entornos de desarrollo (IDEs) para programar en las tecnologías HTML, JavaScript y CSS:

- ▶ [IDEs de JetBrains](#)
- ▶ [Visual Studio Code](#)
- ▶ [Sublime Text](#)
- ▶ [Atom](#)
- ▶ [Brackets](#)

Para que un archivo HTML sea considerado un archivo HTML 5 correcto precisa una mínima estructura, que se compone de doctype, un head, y body.

Tema 4. Lenguajes de marcado. Información de sus etiquetas



Ilustración 8: ejemplo de documento HTML 5.

Existe multitud de etiquetas HTML, cada una para representar un tipo de contenido distinto.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

🔍 Search 113 HTML elements

☒ experimental ☒ meta ☒ self-closing ☒ inline ☒ block

a	meta	self-closing	inline	block
abbr	meta	self-closing	inline	block
address	meta	self-closing	inline	block
area	meta	self-closing	inline	block
article	meta	self-closing	inline	block
aside	meta	self-closing	inline	block
audio	meta	self-closing	inline	block
b	meta	self-closing	inline	block

Ilustración 9: etiquetas HTML 5. Fuente: <https://htmlreference.io/>

Codificación

En informática toda la información se almacena en formato numérico. Para guardar caracteres (letras, números, signos de puntuación, diferentes lenguajes, símbolos, emojis, etc.) es necesario establecer una correspondencia entre los códigos numéricos y los caracteres. Al conjunto de correspondencias se le llama **juego de caracteres**.

Para que todos los ordenadores puedan entender los caracteres de la misma forma y las páginas web puedan ser correctamente interpretadas por igual se emplea un **juego de caracteres estándar**. Unicode es un conjunto de juegos de caracteres estándar utilizado en la actualidad y existen diferentes codificaciones para transformar los caracteres a binario: UTF-8, UTF-16, UTF-32...

Desde 2014 la recomendación HTML 5 especifica que **UTF-8** (Unicode Transformation Format) es la codificación predeterminada.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Los juegos de caracteres traducen caracteres a números. Las codificaciones traducen números a binario.

Para representar la codificación UTF-8 en las páginas HTML simplemente utilizamos la etiqueta **meta** *charset*.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Document</title>
</head>
<body>
```

Ilustración 10: Codificación UTF-8.

Entidades carácter

Para poder mostrar caracteres especiales en HTML utilizamos las entidades de carácter, las cuales son referencias a caracteres.



HTML Entity	Rendered Character
&	&
€	€
©	©
®	®
∀	∀
∈	∈
∉	∉
∑	Σ
´	á
Á	Á
î	í
Ô	Ó
ú	ú

Ilustración 11: Entidades carácter.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Entidades numéricas

Para poder mostrar caracteres especiales en HTML también es posible utilizar entidades numéricas las cuales son referencias a caracteres equivalentes al número Unicode que referencia el carácter a mostrar.

<pre><p>Mostramos el símbolo ampersand: &#38;</p> <p>Mostramos el símbolo euro: &#8364;</p> <p>Mostramos el símbolo copyright: &#169;</p> <p>Mostramos el símbolo marca registrada: &#174;</p></pre>	Mostramos el símbolo ampersand: & Mostramos el símbolo euro: € Mostramos el símbolo copyright: © Mostramos el símbolo marca registrada: ®
--	---

Ilustración 12: Entidades numéricas.

Accesibilidad

La **accesibilidad** de una web o interfaz indica la capacidad de acceso a la misma y a sus contenidos por todas las personas, independientemente de sus conocimientos y de si presentan algún tipo de discapacidad (física, intelectual o técnica).

La accesibilidad web hace referencia al conjunto de prácticas destinadas a lograr que las páginas web sean utilizables por el máximo número de personas.

Algunas **prácticas** utilizadas para lograr la accesibilidad web son:

- ▶ Subtítulos en los vídeos.
- ▶ Imágenes o archivos multimedia que sustituyan o complementen textos.
- ▶ Tamaño fuente variable, para ajustarse a problemas de visión.
- ▶ Tamaño de los elementos en pantallas táctiles.
- ▶ Posibilidad de interacción por voz
- ▶ Posibilidad de reproducción del contenido textual mediante voz
- ▶ Ajustes en los colores

Tema 4. Lenguajes de marcado. Información de sus etiquetas



Ilustración 13: Imagen accesibilidad web. Fuente: dominio público.

Algunos recursos y proyectos dedicados a la mejora de la accesibilidad de las páginas web son:

- ▶ ARIA
- ▶ Web Content Accessibility Guidelines (WCAG)
- ▶ Web Accessibility in Mind (WebAIM)
- ▶ A11Y Style Guide

Usabilidad

La **usabilidad** web hace referencia al conjunto de técnicas que ayudan a los seres humanos a realizar tareas en entornos gráficos por ordenador.

Para que una interfaz gráfica se use debe permitir realizar de forma sencilla e intuitiva cualquier acción a los usuarios de esta.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

La disposición de los elementos, la presentación de estos (aspecto visual), las secuencias de pasos a seguir para realizar cualquier acción son elementos cruciales para proporcionar una buena usabilidad web.

Una mayor usabilidad se traduce en una mejor experiencia de usuario y en consecuencia en navegaciones web satisfactorias, lo que fomenta que más gente utilice la web. Algunos principios que deben seguir las webs son:

- ▶ **Anticipación:** el sitio web debe anticiparse a las necesidades del usuario.
- ▶ **Autonomía:** la interfaz debe ser lo suficientemente intuitiva como para que los usuarios tengan el control del sitio web sin necesidad de ayuda o indicaciones constantes.
- ▶ **Colores:** la paleta de colores utilizada debe estar en armonía y facilitar la visualización en lugar de complicarla o dispersar la atención.
- ▶ **Reversibilidad:** permitir deshacer acciones realizadas
- ▶ **Ley de Fitts:** el tiempo para alcanzar un objetivo con el ratón está en función de la distancia y el tamaño del objetivo, estos deben estar lo más cerca de la posición previa.

Validación HTML

Existen diversas herramientas online con las que verificar que el marcado HTML que hemos realizado es correcto y no tiene errores como, por ejemplo: etiquetas mal cerradas, etiquetas mal escritas, orden de etiquetas incorrecto, etc.

[W3C Markup Validation Service](#) es una de ellas.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

HTML semántico

HTML semántico hace referencia a utilizar los elementos HTML correctos para cada propósito específico.

Por ejemplo, a la hora de definir un botón, en vez de utilizar un elemento genérico como puede ser `<div>`, mejor utilizar una etiqueta semántica que ya esté creada para botones, como es el caso de `<button>`.

Elementos **genéricos**:

`<div>`, `<span>`

Elementos **semánticos**:

`<button>`, `<form>`, `<table>`, `<h1>`, `<article>`, `<footer>`, `<nav>`, `<header>`

Los motores de búsqueda indexan mejor la información cuando utilizamos elementos semánticos, favoreciendo el SEO.

Otro aspecto semántico importante son los **atributos**. El atributo **alt** permite ofrecer un texto descriptivo para una imagen en caso de que la imagen no pueda ser cargada en el navegador por problemas de conexión lenta.

``

Tema 4. Lenguajes de marcado. Información de sus etiquetas

El atributo **title** sirve para añadir información extra a un elemento, de manera que cuando pasamos por encima con el ratón se muestra en forma de tooltip. Se emplea especialmente en enlaces:

Accede pulsando `<a href="/enlace" title="Enlace a Intranet">aquí</a>`

Tema 4. Lenguajes de marcado. Información de sus etiquetas

4.5. XML



Ilustración 14: Logo lenguaje de marcado XML. Fuente: dominio público.

XML es el **lenguaje de marcado extensible**. Es un estándar aprobado por W3C para el marcado de documentos. Define una sintaxis genérica utilizada para marcar datos con etiquetas legibles por humanos. Proporciona un formato estándar para documentos informáticos que es lo suficientemente flexible como para personalizarse para dominios tan diversos como sitios web, intercambio de datos, gráficos vectoriales, genealogía, serialización de objetos, llamadas a procedimientos remotos, sistemas de correo de voz y más.

Es posible crear programas informáticos que lean y manipulen datos en documentos XML. Existe una amplia gama de librerías en una variedad de lenguajes de programación que pueden leer y escribir XML.

El software estándar, como navegadores web y editores de texto, tienen la capacidad de trabajar con documentos XML. Algunas de herramientas pueden trabajar con cualquier documento XML. Otras están personalizadas para admitir una aplicación XML particular en un dominio particular, como gráficos vectoriales, pero la misma sintaxis subyacente es utilizada en todos los casos.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

XML es un lenguaje de meta marcado para documentos de texto. Los datos están incluidos en XML en documentos como cadenas de texto. Los datos están rodeados por marcas de texto que describen los datos. La unidad básica de datos y marcado de XML se denomina **elemento**.

La especificación XML define la sintaxis exacta que debe seguir este marcado:

- ▶ Cómo los elementos están delimitados por etiquetas.
- ▶ Qué aspecto tiene una etiqueta.
- ▶ Qué nombres son aceptables para los elementos.
- ▶ Dónde se colocan los atributos.

Superficialmente, el marcado en un documento XML se parece mucho al marcado de un documento HTML, pero existen diferencias importantes.

Lo más importante es que XML es un lenguaje de meta marcado. Eso significa que no tiene conjunto fijo de etiquetas y elementos que se supone que funcionan para todos en todas las áreas de interés para todos los tiempos como ocurre en HTML.

XML permite inventar nuevos elementos según sean necesarios:

- ▶ Los químicos pueden usar elementos que describen moléculas, átomos, enlaces, reacciones y otros elementos encontrados en química.
- ▶ En el sector inmobiliario se pueden utilizar elementos que describen apartamentos, alquileres, comisiones, ubicaciones, y otros artículos necesarios para bienes raíces.
- ▶ Los músicos pueden usar elementos que describen negritas, medias notas, claves de sol, letras y otros objetos comunes en la música.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

La X en XML significa **Extensible**. Extensible significa que el idioma puede ser ampliado y adaptado para satisfacer muchas necesidades diferentes.

Aunque XML es bastante flexible en los elementos que permite, es bastante estricto en muchos otros aspectos. La especificación XML define una gramática para documentos XML que dice dónde se pueden colocar las etiquetas, cómo deben verse, qué elementos y nombres son legales, cómo se adjuntan los atributos a los elementos, etc.

Esta gramática es lo suficientemente específica como para permitir el desarrollo de analizadores XML que pueden leer cualquier documento XML. Los documentos que satisfacen esta gramática son documentos XML bien formados.

No se permiten documentos que no estén bien formados, como tampoco se permite un programa en Java que contiene un error de sintaxis. Los procesadores XML rechazan documentos que contienen errores de forma correcta.

Por razones de interoperabilidad, las personas u organizaciones pueden acordar utilizar solo ciertas etiquetas. Estos conjuntos de etiquetas se denominan aplicaciones XML. Una aplicación XML es no es una aplicación de software que utilice XML, como Mozilla o Microsoft Word. Es una aplicación de XML en un dominio particular, como gráficos vectoriales (SVG).

Marcado

El **marcado** en un documento XML describe la estructura del documento.

Permite ver qué elementos están asociados con qué otros elementos. En un documento XML bien diseñado, el marcado también describe la semántica del documento. Por ejemplo, el marcado puede indicar que un elemento es una fecha o una persona o un código de barras.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

En aplicaciones XML bien diseñadas, el marcado no dice nada sobre cómo debe mostrarse el documento. Es decir, no dice que un elemento esté en negrita o en cursiva o es un elemento de una lista. XML es un lenguaje de marcado estructural y semántico, no un lenguaje de marcado de tipo presentación.

El marcado permitido en una aplicación XML en particular se puede documentar en un **esquema**. Las instancias de documentos particulares se pueden comparar con el esquema. Se dice que los documentos que coinciden con el esquema son **válidos**.

Los documentos que no coinciden no son válidos. La validez depende del esquema. Es decir, si un documento es correcto o incorrecto depende del esquema con el que lo compare. No todos los documentos necesitan ser validados. Para muchos propósitos es suficiente que el documento esté bien formado.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

4.6. XHTML



Ilustración 15: Logo lenguaje de marcado XHTML. Fuente: dominio público.

XHTML (Extensible HyperText Markup Language), desarrollado por World Wide Consortium (W3C) en 2000, es la versión más extendida de HTML.

Es el mismo HTML, pero definido como una aplicación XML, que combina las ventajas de HTML y XML.

XHTML conserva todas las características de HTML, pero introduce reglas más estrictas para la creación de páginas, lo que le permite hacer que los sitios sean independientes del dispositivo de visualización y del navegador. Significa que el sitio se mostrará correctamente en todos los navegadores y plataformas modernos como ordenadores, tabletas, smartphones, etc.

Diferencias entre HTML y XHTML

- Siempre debe usar `<!DOCTYPE ...>`. Un documento XHTML debe incluir una declaración de tipo de documento XHTML. Otros elementos obligatorios son las etiquetas `<html>`, `<head>`, `<title>` y `<body>`.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

- ▶ Se deben insertar elementos XHTML correctamente. Un documento XHTML, no permite anidar elementos incorrectamente.
- ▶ A diferencia de HTML, donde a veces es posible omitir la etiqueta de cierre, en XHTML la etiqueta de cierre se requiere siempre y en todas partes.
- ▶ Las etiquetas en minúsculas y mayúsculas se distinguen en XHTML. Para evitar confusiones, todas las etiquetas, así como sus atributos, deben escribirse en minúsculas.
- ▶ Los valores de los atributos deben ir entre comillas.
- ▶ No se puede minimizar los atributos. Al atributo abreviado no se le asigna ningún valor. A un atributo se le puede asignar un valor de atributo que coincida con el nombre. Ejemplo:

Mal:

```
<input type="checkbox" name="flower" value="rose" checked />
```

Bien:

```
<input type="checkbox" name="flower" value="rose" checked="checked" />
```

- ▶ En lugar del atributo de nombre, es mejor que use id. En XHTML, el nombre está parcialmente obsoleto, por eso se recomienda usar el atributo id.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Beneficios de XHTML

El hecho de que en XHTML todas las etiquetas deban cerrarse y anidarse correctamente hace que el código se vea mucho más limpio.

XHTML usa menos ancho de banda, lo que optimiza la carga del documento XHTML en los navegadores.

Gracias a su buen formato, los documentos XHTML se transportan fácilmente a dispositivos inalámbricos, lectores de Braille y otros entornos web especializados.

Al igual que con HTML normal, XHTML funciona con CSS para crear páginas web fácilmente.

XHTML vs. HTML

HTML es el lenguaje de marcado principal para crear páginas web, mientras que XHTML, es la versión extendida del mismo.

HTML es una aplicación del lenguaje de marcado estándar generalizado (SGML), mientras que XHTML es una aplicación de XML.

HTML se amplía desde SGML y XHTML desde XML y HTML. HTML fue creado por Tim Berners-Lee entre 1980 y 1989 y XHTML fue propuesto por World Wide Consortium en 2000.

La última diferencia entre estos dos es el marco flexible que requiere un analizador específico de HTML permisible en el caso de HTML y un subconjunto restrictivo de XML que debe analizarse con un analizador XML estándar en el caso de XHTML.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

XHTML vs HTML 5

XHTML distingue entre mayúsculas y minúsculas en sus etiquetas, mientras que HTML5, así como HTML, no son sensibles a mayúsculas y minúsculas.

La siguiente diferencia es la compatibilidad del navegador: XHTML es compatible con todos los navegadores, mientras que HTML 5 tiene ciertas etiquetas semánticas que no son compatibles en versiones antiguas de algunos navegadores.

HTML5 es más adecuado para dispositivos móviles, mientras que XHTML es más adecuado para pantallas de escritorio. HTML5 es más permisible que XHTML a la hora de escribir las etiquetas y elementos.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

4.7. LaTeX



Ilustración 16: Logo lenguaje de marcado LATEX. Fuente: dominio público.

Donald E. Knuth desarrolló **TEX** en el año 1977 como un sistema de composición tipográfica para preparar libros, especialmente aquellos que contienen expresiones matemáticas.

Basado en él, Leslie Lamport desarrolló **LATEX** (llamado LATEX 2.09) en 1985 para preparar documentos permitiendo enfocarse en la estructura de un documento en lugar de sus detalles de formato.

LATEX consiste en un paquete de macros, un **sistema de preparación de documentos** para una composición tipográfica de alta calidad. Se utiliza con mayor frecuencia para **documentos técnicos o científicos** de tamaño medio a grande, pero se puede utilizar para casi cualquier forma de publicación. Permite compilar un archivo en formato .text a otros formatos como pdf.

LATEX se puede utilizar para preparar cartas, aplicaciones, artículos, informes, publicaciones, tesis, libros, documentos de química, física, computación, biología, leyes, literatura, música y cualquier otra temática.

LATEX tiene la posibilidad de **generar automáticamente** varias listas de contenidos, índices, glosario, numerar capítulos y figuras, incluir y organizar la bibliografía

Tema 4. Lenguajes de marcado. Información de sus etiquetas

adecuada, mantener índices y referencias cruzadas, etc. Por tanto, la posibilidad de cometer algún error en la numeración y referencia de elementos (apartados, tablas, figuras o ecuaciones), al elegir el tamaño y el tipo de fuente para las diferentes secciones y subsecciones, o en la preparación de referencias bibliográficas, puede evitarse.

Las **operaciones principales** para crear documentos en LATEX son: editar, compilar, y visualizar:

- ▶ **Editar:** utilizar un editor o IDE para generar un archivo, con terminación .tex, que contiene el código en LATEX describiendo la estructura y el contenido del documento.
- ▶ **Compilar.** Compilar es el proceso, realizado por el motor de LATEX, que convierte los archivos .tex en documentos con formato .pdf que se pueden imprimir y ver en pantalla.
- ▶ **Visualizar.** Una vez compilado el documento, si no hubo errores, se puede visualizar el documento generado por LATEX. En [TeXnicCenter](#), por ejemplo, hay un botón que permite iniciar el visualizador de documentos.

Entre las **ventajas** de LATEX se encuentran:

- ▶ LATEX es ampliamente utilizado en entornos científicos. Muchas revistas aceptan documentos escritos en LATEX.
- ▶ Excelente calidad del documento final con salida en distintos formatos: dvi, pdf, ps, etc.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

- ▶ Los ficheros fuente .tex son ficheros ASCII y pueden ser compilados en cualquier sistema operativo.
- ▶ Es gratuito.
- ▶ Muy potente.

Como **desventajas** destaca:

No es un procesador del tipo **wysiwyg** (*What You See Is What You Get*) «lo que ves es lo que consigues», por lo que es necesario un proceso de compilación (con posibles errores en caso de que algo esté mal escrito).

LATEX no es un procesador de textos, permite a los autores evitar preocuparse por la apariencia de los documentos, sino concentrarse en obtener el contenido correcto.

Ejemplo de código LATEX:

```
\documentclass{article}

\title{Cartesian closed categories and the price of eggs}

\author{Jane Doe}

\date{September 1994}

\begin{document}

\maketitle

Hello world!

\end{document}
```

Tema 4. Lenguajes de marcado. Información de sus etiquetas

4.8. Markdown



Ilustración 17: Logo lenguaje de marcado ligero Markdown. Fuente: dominio público.

Markdown es un lenguaje de marcado ligero que puede usar para agregar elementos de formato a documentos de texto sin formato. Markdown, creado por John Gruber en 2004, es ahora uno de los lenguajes de marcado más populares del mundo.

Cuando se crea un archivo con formato de Markdown, se agrega la sintaxis de Markdown al texto para indicar qué palabras y frases deben verse diferentes:

- ▶ **Encabezado:** se agrega un signo de número antes de él (por ejemplo, # Encabezado uno).
- ▶ Una frase en **negrita** agregue dos asteriscos antes y después (por ejemplo, ** este texto está en negrita **).

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Markdown no es un lenguaje **WYSIWYG**. Según su creador, la sintaxis de Markdown está diseñada para ser legible y simple, por lo que el texto en los archivos de Markdown se puede leer incluso si no se procesa, es por eso por lo que es considerado un lenguaje de marcado ligero.

Uso de Markdown

Entre los principales usos de Markdown se encuentran:

- ▶ Contenido para sitios web estáticos autogenerados ([Jekyll](#), [GatsbyJS](#), Hugo)
- ▶ Documentos
- ▶ Notas
- ▶ Libros
- ▶ Presentaciones
- ▶ Mensajes de correo electrónico
- ▶ Documentación técnica

Como ventajas de Markdown destaca que es **portable**, a diferencia de las aplicaciones de procesamiento de texto tradicionales como Microsoft Word que bloquean su contenido en un formato de archivo propietario.

Markdown es **independiente de la plataforma**. Puede crear texto con formato Markdown en cualquier dispositivo que ejecute cualquier sistema operativo.

Markdown se utiliza en las principales plataformas colaborativas de internet. Los sitios web como [Reddit](#) y [GitHub](#) admiten Markdown, y muchas aplicaciones de escritorio y basadas en la web lo admiten.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

¿Cómo funciona markdown?

Cuando se escribe un documento en Markdown, el texto se almacena en un archivo de texto sin formato que tiene una **extensión .md** o markdown. Para visualizar el archivo Markdown es necesaria una aplicación capaz de renderizar el archivo.

Las aplicaciones de Markdown utilizan algo llamado **procesador Markdown** (también conocido como "analizador" o "implementación") para tomar el texto con formato Markdown y enviarlo a formato HTML de manera que pueda verse en un navegador web o combinarse con una hoja de estilo e imprimirse.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

4.9. Otras tecnologías

JSON

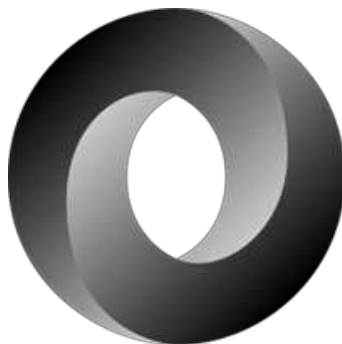


Ilustración 18: Logo lenguaje de intercambio de información JSON. Fuente: dominio público.

JSON es un **formato de intercambio de datos**. Los formatos para el intercambio de datos son formatos de texto que se utilizan para intercambiar datos entre plataformas software.

Las aplicaciones informáticas necesitan formatos de intercambio de datos, como XML y JSON, para intercambiar datos entre diferentes sistemas informáticos.

La capacidad de estos sistemas únicos para comunicarse es parte integral de muchas empresas y organizaciones. Si cada uno de estos sistemas necesitara un traductor para todas las formas en que otros sistemas estructuran sus datos, entonces las comunicaciones consumirían una cantidad adecuada de tiempo y recursos.

Para evitar esa problemática, los sistemas acuerdan un formato único para obtener datos y emplear un solo traductor. JSON es un formato de intercambio de datos que muchos sistemas han acordado utilizar para la comunicar datos entre sí.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Muchos, pero no todos los sistemas, han acordado JSON para comunicar datos. Existen formatos de intercambio de datos, como Extensible Markup Language (XML), que fueron antes de que siquiera se popularizara JSON.

Muchos sistemas tienen y siguen utilizando otros formatos, como XML o formatos más tabulares y delimitados, como valores separados por comas (CSV).

JSON son las siglas de **JavaScript Object Notation**. El nombre de esta tecnología para el intercambio de datos puede inducir a error a las personas a pensar que necesitarán aprender JavaScript para poder utilizar JSON.

No es necesario aprender JavaScript para poder utilizar JSON, pero cabe destacar que su gran popularidad viene conferida gracias a que el lenguaje JavaScript lo interpreta nativamente. Eso significa que JSON es un formato idóneo para intercambiar datos entre el backend y el frontend de una aplicación web.

El **frontend** de una aplicación web puede ser una aplicación desarrollada en JavaScript vanilla o con algún framework (**Angular, React, Vue**) y recibir datos en formato JSON desde una **API REST** en el **backend**. Al ser JSON un formato nativo para JavaScript, no se pierde tiempo en conversión de formatos para poder utilizar los datos desde JavaScript.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

YAML



Ilustración 19: Logo lenguaje de marcado XHTML. Fuente: dominio público.

YAML es un lenguaje de serialización de datos que se usa a menudo para escribir archivos de configuración. YAML puede considerarse otro lenguaje de marcado pero en general YAML es para datos, no para documentos. YAML se utiliza en conjunto a otros lenguajes de programación.

Sintaxis

YAML tiene características que provienen de Perl, C, XML, HTML y otros lenguajes de programación. YAML también es un superconjunto de JSON, por lo que los archivos JSON son válidos en YAML.

YAML usa sangría estilo Python para indicar anidamiento. Los caracteres de tabulación no están permitidos, por lo que se utilizan espacios en blanco en su lugar. No hay símbolos de formato habituales, como llaves, corchetes, etiquetas de cierre o comillas. Los archivos YAML usan una extensión **.yml** o **.yaml**.

La estructura de un archivo YAML es un mapa o una lista:

- ▶ Los mapas le permiten asociar pares clave-valor. Cada clave debe ser única y el orden no importa. Piense en un diccionario de Python o una asignación de variable en un script Bash.
- ▶ Un mapa en YAML debe resolverse antes de que se pueda cerrar y se crea un nuevo mapa. Se puede crear un nuevo mapa aumentando el nivel de sangría o resolviendo el mapa anterior e iniciando un mapa adyacente.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

- ▶ Una lista incluye valores enumerados en un orden específico y puede contener cualquier número de elementos necesarios. Una secuencia de lista comienza con un guión (-) y un espacio, mientras que la sangría la separa del padre. Puede pensar en una secuencia como una lista de Python o una matriz en Bash o Perl. Se puede incrustar una lista en un mapa.

YAML también contiene escalares, que son datos arbitrarios (codificados en Unicode) que se pueden usar como valores como cadenas, enteros, fechas, números o booleanos.

Al crear un archivo YAML, se deben seguir unas reglas de sintaxis de manera que los archivos sean válidos. Un linter es una aplicación que verifica la sintaxis de un archivo. El comando `yamllint` puede ayudar a garantizar que haya creado un archivo YAML válido antes de entregarlo a una aplicación.

Usos de YAML

Uno de los usos más comunes de YAML es crear archivos de configuración. Se recomienda que los archivos de configuración se escriban en YAML en lugar de JSON, aunque se pueden usar indistintamente en la mayoría de los casos, porque YAML tiene una mejor legibilidad y es más fácil de usar.

YAML también es utilizado por la herramienta de automatización para sistemas, como es el caso de las configuraciones de las herramientas:

- ▶ Ansible para crear procesos de automatización
- ▶ Recursos e implementaciones de Kubernetes.

Una ventaja de usar YAML es que los archivos YAML se pueden agregar al control de código fuente, como GitHub, para que los cambios se puedan rastrear y auditar.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

Ansible

Los Playbooks de **Ansible** se utilizan para orquestar los procesos de TI. Un Playbook en Ansible es un archivo YAML que contiene 1 o más reproducciones y se usa para definir el estado deseado de un sistema.

Cada Playbook puede ejecutar una o más tareas, y cada tarea invoca un módulo de Ansible. Los módulos se utilizan para realizar tareas de automatización en Ansible. Los módulos Ansible se pueden escribir en cualquier lenguaje que pueda devolver JSON, como Ruby, Python o bash.

Un Playbook de Ansible consta de mapas y listas. Para crear un Playbook, se inicia una lista YAML que nombre la obra y luego enumere las tareas en una secuencia. La sangría no es una indicación de herencia lógica. Es necesario pensar cada línea como un tipo de datos YAML (una lista o un mapa).

Mediante el uso de plantillas YAML, en Ansible se pueden programar tareas repetitivas para que sucedan automáticamente sin tener que aprender un lenguaje de programación avanzado.

Kubernetes

Kubernetes funciona según el estado definido y el estado real. Los objetos de Kubernetes representan el estado de un clúster y le dicen a Kubernetes cómo desea que se vea la carga de trabajo. Los recursos de Kubernetes, como pods, objetos e implementaciones, se pueden crear con archivos YAML.

Al crear un objeto de Kubernetes, deberá incluir especificaciones para definir el estado deseado del objeto. La API de Kubernetes se puede utilizar para crear el objeto. La solicitud a la API incluirá las especificaciones del objeto en JSON, pero la mayoría de las veces proporcionará la información requerida a kubectl como un archivo YAML.

Tema 4. Lenguajes de marcado. Información de sus etiquetas

KubectI convertirá el archivo a YAML cuando realice la solicitud de API. Una vez que se ha creado y definido un objeto, Kubernetes trabaja para asegurarse de que el objeto siempre exista.

Los desarrolladores o administradores de sistemas especifican el estado definido mediante los archivos YAML o JSON que envían a la API de Kubernetes. Kubernetes usa un controlador para analizar la diferencia entre el nuevo estado definido y el estado real en el clúster.